



Git



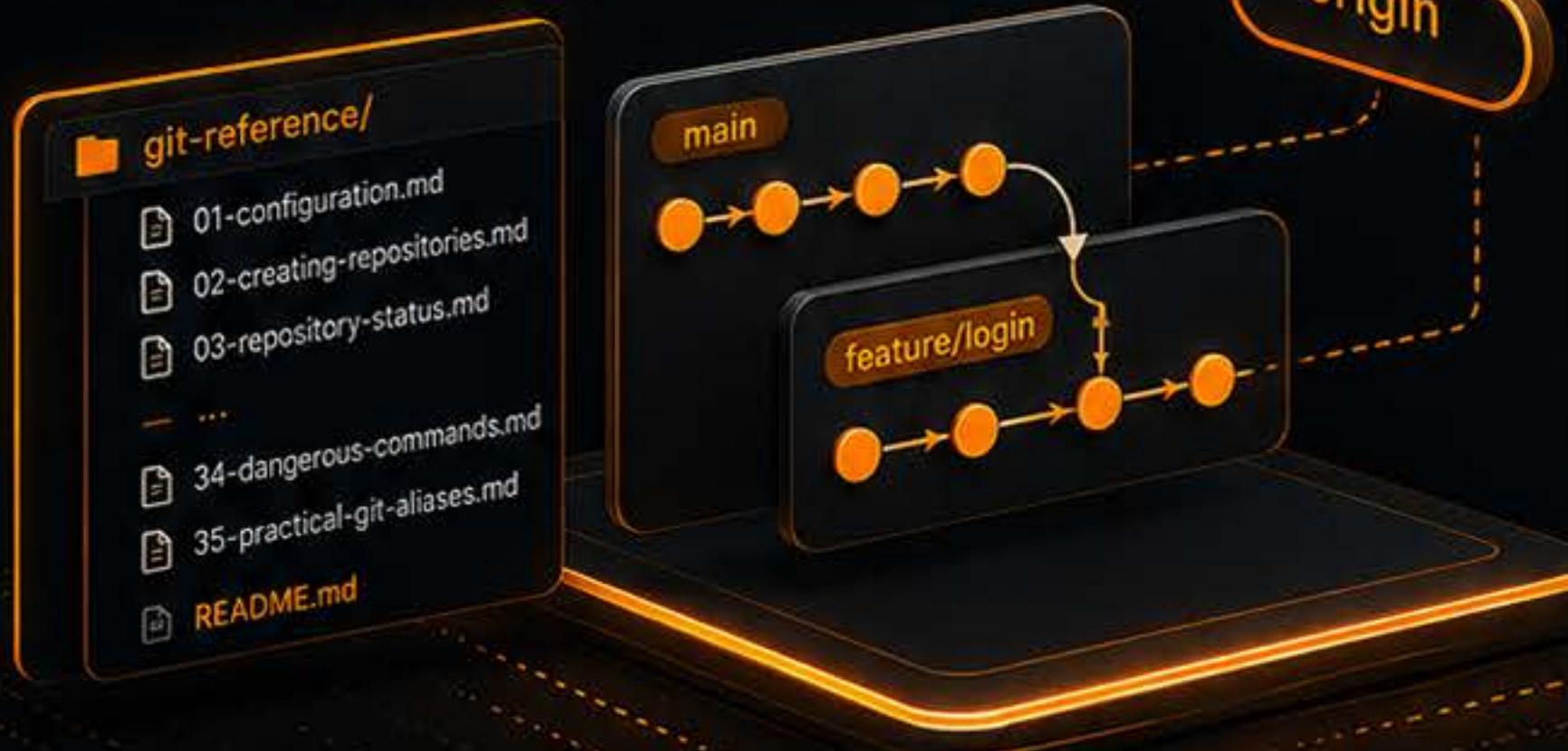
35

CHAPTERS



Command Reference

Your complete **Git command reference** for Developers, Software Engineers and DevOps Engineers.



PRACTICAL COMMANDS
with examples and explanations



WORKFLOWS
Developer & DevOps best practices



RECOVERY & SAFETY
reflog, recovery and dangerous commands



AUTOMATION & DEVOPS
CI/CD, hooks, scripts and more



ALIASES & SHORTCUTS
boost your productivity every day

GIT WORKFLOW



WORKING TREE



STAGING AREA



COMMIT (REPOSITORY)



REMOTE REPOSITORY



CODE REVIEW



MERGE



DEVELOPERS
Daily commands you use every day



SOFTWARE ENGINEERS
Advanced Git techniques and history analysis



DEVOPS ENGINEERS
CI/CD, automation and infrastructure workflows



GIT INTERNALS
Objects, refs, .git directory and more



SAFETY FIRST
Understand the impact before you run



MEMORIZE WHAT MATTERS
High-value commands and quick reference



1. CONFIGURATION

Fine-tune Git to work **your** way.

Control identity, behavior, defaults, aliases, tools, credentials, and repository-specific settings.



IDENTITY



BEHAVIOR



DEFAULTS



ALIASES



CREDENTIALS



MERGE / REBASE
PREFERENCES



REPO-SPECIFIC
SETTINGS

CONFIGURATION LEVELS



SYSTEM

All users on the machine



GLOBAL

Current user



LOCAL

Current repository



WORKTREE

Specific worktree



COMMAND-LINE

Highest precedence



More specific settings override more general ones.

EXAMPLE: VIEW ALL CONFIGURATION

```
$ git config --list  
user.name=Marko  
user.email=dev@example.com  
init.defaultbranch=main  
pull.rebase=true  
core.editor=vim  
core.autocrlf=input  
alias.st=status -sb  
...
```

★ KEY BENEFITS

- ✓ Consistent workflow
- ✓ Fewer errors
- ✓ Better productivity
- ✓ Team alignment
- ✓ Safer operations

POPULAR CONFIGURATION HIGHLIGHTS



`pull.rebase=true`
Pull with rebase



`push.autoSetupRemote=true`
Auto upstream



`init.defaultBranch=main`
Use main by default



`core.editor=vim`
Set your editor



`core.autocrlf=input`
Handle line endings



`alias.st=status -sb`
Aliases save time



ESSENTIAL CONFIGURATION CHECKLIST

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"  
git config --global init.defaultBranch main  
git config --global pull.rebase true  
git config --global push.autoSetupRemote true  
git config --global core.autocrlf input
```



NO BRANCH IMPACT

Configuration commands don't change your branch or working tree.

They change how Git behaves.

CONFIGURE ONCE. WORK FASTER. **STAY IN CONTROL.**



2. CREATING REPOSITORIES

Create. Clone. Configure. **Ship.**

Everything you need to start a Git repository the right way.

TWO WAYS TO GET A GIT REPOSITORY

1. INITIALIZE NEW REPOSITORY



```
$ git init -b main
```

✓ Creates a new repository from scratch

2. CLONE EXISTING REPOSITORY



```
$ git clone <repository-url>
```

✓ Get a full copy of an existing repository

ESSENTIAL COMMANDS



git init
Initialize new repository



git init -b main
Initialize **with** specific branch



git clone <url>
Clone repository



git clone -b <branch> <url>
Clone specific branch



git clone --depth 1 <url>
Shallow clone (limited history)



git clone --no-checkout <url>
Clone without checking out files



git init --bare
Create bare (repository only)



git remote add origin <url>
Connect local repo to remote



git push -u origin main
Push and set upstream



git status
Verify repository status

BARE vs NON-BARE

NORMAL REPO



Working tree + .git/
For development

BARE REPO



No working tree (Git data only)
For servers & remotes

VS

COMMON WORKFLOWS

- ⊕ Create new project ➡ 🚀
- ⊕ Connect to remote ➡ ☁️
- ⊕ Clone existing repo ➡ 📁
- 🕒 Shallow clone for CI ➡ 📁
- 🗄️ Mirror / backup ➡ 🗄️

QUICK VERIFICATION

- ✓ **git branch --show-current**
- ✓ **git remote -v**
- ✓ **git rev-parse --show-toplevel**
- ✓ **git rev-parse --git-dir**
- ✓ **git rev-parse --is-inside-work-tree**



FOUNDATION OF EVERY GREAT PROJECT.

Master repository creation and you're ready to build, collaborate, and scale.





3. REPOSITORY STATUS & INFORMATION

Inspect **everything**. Understand your repository.



WORKING
TREE



STAGING
AREA



REPOSITORY



REMOTE



BRANCHES



TAGS



OBJECTS



INFO

GIT STATUS – YOUR FIRST STOP



```
$ git status -sb
## main...origin/main [ahead 2, behind 1]
M  src/app.js
A  src/config.js
?? debug.log
```

M Modified

A Added

D Deleted

MM Modified (staged & unstaged)

?? Untracked

XY X = staged, Y = working tree

KEY AREAS YOU CAN INSPECT



Branches

`git branch -vv`



History

`git log --oneline --graph
--decorate --all`



Remotes

`git remote -v`



Commits & Objects

`git show HEAD
git cat-file -t HEAD`



Repository Paths

`git rev-parse --show-toplevel
--git-dir`



Upstream Tracking

`git rev-parse --abbrev-ref
@{upstream}`



Diagnostics

`git count-objects -v
git fsck --full`

POWERFUL INSPECTION COMMANDS



`git status -sb`
Quick overview



`git branch -a`
All branches



`git log --graph
--decorate --all`
Full history graph



`git remote -v`
Remote info



`git tag`
List tags



`git rev-parse
--short HEAD`
Current commit



`git fsck --full`
Check integrity

DEVELOPER INSPECTION FLOW

- | | |
|--------------------------|---|
| 1 Where am I? |  <code>rev-parse --show-toplevel</code> |
| 2 What branch? |  <code>branch --show-current</code> |
| 3 What changed? |  <code>status -sb</code> |
| 4 What's the history? |  <code>log --graph --decorate --all</code> |
| 5 Where is it going? |  <code>branch -vv</code> |
| 6 What's connected? |  <code>remote -v</code> |
| 7 Is everything healthy? |  <code>fsck --full</code> |



WHY THIS MATTERS

- ✓ Know your repository state instantly
- ✓ Navigate branches with confidence
- ✓ Understand what changed and why
- ✓ Track remotes and upstream status
- ✓ Diagnose issues before they bite
- ✓ Essential for DevOps and CI/CD
- ✓ Read-only commands. Zero risk.



KNOW YOUR REPO. STAY IN CONTROL.

From quick status to deep diagnostics – every developer's essential toolkit.



4. STAGING & COMMITTING

From changes to **history**.

Stage the right changes. Commit with confidence.

WORKING TREE



Your files
change here

STAGING AREA / INDEX



Choose what goes
into the next commit

REPOSITORY HISTORY



Permanent, versioned
history

REMOTE REPOSITORY



Share and
collaborate

`git add`

`git commit`

`git push`

STAGE SMART



`git add <file>`
Stage a specific file



`git add <dir>/`
Stage a directory



`git add -A`
Stage all changes



`git add -i`
Interactive staging



`git add -p`
Stage by hunk (patch)

INSPECT & REVIEW



`git status -sb`
Quick overview



`git diff`
Unstaged changes



`git diff --cached`
Staged changes



`git diff --cached --stat`
Staged statistics



`git status --porcelain`
Script-friendly output

COMMIT WITH CONFIDENCE



`git commit -m "message"`
Commit with a clear message



`git commit -am "msg"`
Stage tracked changes
and commit



`git commit --amend`
Fix the last commit



`git commit -S`
Sign your commits



`git commit --allow-empty`
Create an empty commit

WHAT DOES EACH SYMBOL MEAN?

M Modified (unstaged)	D Deleted (staged)
m Modified (staged)	?? Untracked
MM Modified (staged & unstaged)	R Renamed
A Added (staged)	XY X = staged, Y = working tree

TYPICAL STATUS -SB OUTPUT

```
## main...origin/main [ahead 2, behind 1]
M  src/app.js
A  src/config.js
MM src/auth.js
?? debug.log
```

- ← Branch + tracking state
- ← Staged change
- ← Added file (staged)
- ← Staged & unstaged
- ← Untracked file

THE DEVELOPER WORKFLOW



1. CHECK
`git status -sb`



2. STAGE
`git add -p`



3. REVIEW
`git diff --cached`



4. COMMIT
`git commit -m "Describe change"`



5. VERIFY
`git status -sb`

WHY IT MATTERS

- ✓ Clean, focused commits
- ✓ Clear project history
- ✓ Easy reviews & collaboration
- ✓ Safe to push and deploy



GOOD COMMITS TODAY. EASY HISTORY TOMORROW.



5. DIFF & CODE REVIEW

Inspect. Compare. Review. Commit with **confidence**.
See exactly what changed before it becomes history.

WHY IT MATTERS

- ✓ Catch issues early
- ✓ Create clean commits
- ✓ Better code reviews
- ✓ Stronger collaboration
- ✓ CI/CD quality gates

GIT DIFF MODEL

WORKING TREE



Your files change here

STAGING AREA / INDEX



Choose what goes into the next commit

REPOSITORY HISTORY



Permanent, versioned history

REMOTE REPOSITORY



Share and collaborate

git diff

Working Tree ↔ Index
(unstaged changes)

git diff --cached

Index ↔ HEAD
(staged changes)

git diff HEAD

Working Tree ↔ HEAD
(all local changes)

ESSENTIAL DIFF COMMANDS

- git diff** See unstaged changes
- git diff --cached** . See staged changes
- git diff HEAD** See all local changes
- git diff A...B** Compare branches (merge base)
- git diff --stat** . Show change statistics
- git diff --name-status** Files + change status
- git grep 'text'** .. Search inside diffs

DIFF OUTPUT EXAMPLE

```
diff --git a/app.js b/app.js
index 1234567..abcdef0 100644
--- a/app.js
+++ b/app.js
@@ -10,6 +10,7 @@
  const app = express();
- app.listen(PORT);
+ app.use(auth());
  app.listen(PORT);
```

Added Removed Context

STAGE SMART, COMMIT CLEAN

- REVIEW CHANGES**
git diff
- STAGE PRECISELY**
git add -p
- REVIEW STAGED**
git diff --cached
- COMMIT**
git commit -m "..."
- VERIFY**
git show --stat HEAD

POWERFUL REVIEW OPTIONS

-w
Ignore whitespace

--word-diff
Word-level changes

--histogram
Better algorithm for code

-M
Detect renames

-C
Detect copies

--color
Colorized output

--patch
Interactive hunk staging

COMPARE ANYTHING

- Commits: **git diff A B**
- Branches: **git diff main...feature**
- Remote: **git diff origin/main...HEAD**
- Range: **git log -p A...B**

PATCH POWER

- Export: **git diff > changes.patch**
- Apply: **git apply changes.patch**
- Check: **git apply --check changes.patch**
- Reverse: **git apply -R changes.patch**

CI / DEVOPS USE CASES

- ✓ Fail if uncommitted changes exist
- ✓ Generate change summary
- ✓ Verify generated files are clean
- ✓ Drive smart build & test pipelines



REVIEW WITH PRECISION. COMMIT WITH PURPOSE.

Great diffs lead to **great code**.

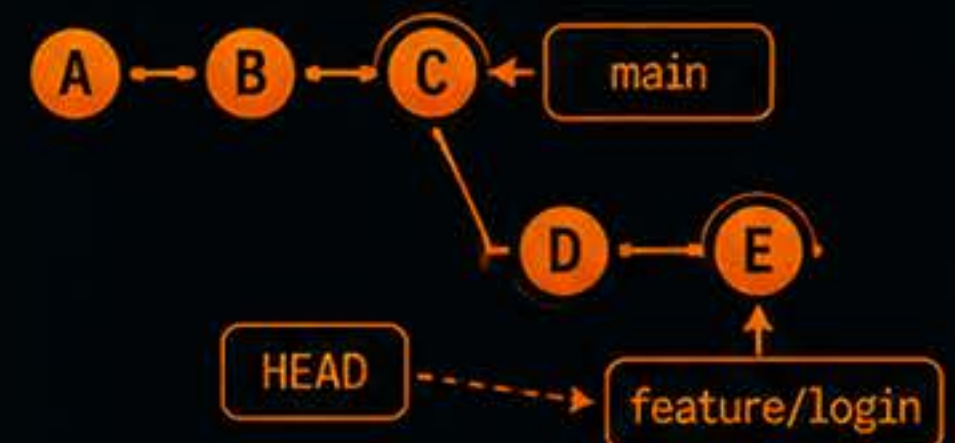


6. BRANCHING

Create. Switch. Manage.
Collaborate.

Branches let you work **isolated**,
merge with **confidence**.

BRANCH MODEL



A branch is a **movable pointer**
to a commit.

CREATE, SWITCH, MANAGE



Create branch
`git branch`
feature/login



Switch branch
`git switch`
feature/login



Create & switch
`git switch -c`
feature/login



Rename branch
`git branch -m`
old new



Delete branch
`git branch -d`
feature/login

BRANCH OVERVIEW



List local
`git branch`



List remote
`git branch -r`



All branches
`git branch -a`



With commits
`git branch -v`



With upstream
`git branch -vv`



`git branch -vv`

```
* main      4cd5678 [origin/main]
develop    7ab1234 [origin/develop]
feature/login 9ef0123 [origin/feature/login:
                                ahead 2, behind 1]
```

● ahead: local has commits upstream lacks
● behind: upstream has commits you lack

TRACKING & UPSTREAM



Set upstream
`git push -u origin`
feature/login



Check upstream
`git rev-parse`
@{upstream}



Divergence
`git rev-list --left-right`
`--count HEAD...@{upstream}`

Example output:

```
2 1
ahead behind
2 commits only on
your branch
1 commit only on
upstream
```

INSPECT & COMPARE



Unstaged changes
`git diff`



Staged changes
`git diff --cached`



All local changes
`git diff HEAD`



Branch vs branch
`git diff main...HEAD`



Statistics
`git diff --stat`

● ● ● main...HEAD

```
@@ -10,6 +10,8 @@
const app = express();

+app.use(auth());
+app.use(rateLimit());

app.listen(PORT);
... and much more
```

COMMON WORKFLOW

1



Update main
`git pull --ff-only`

2



Create feature
`git switch -c feature/x`

3



Make changes
edit files

4



Stage carefully
`git add -p`

5



Review staged
`git diff --cached`

6



Commit
`git commit -m "..."`

7



Push & track
`git push -u origin feature/x`

BRANCH NAMING EXAMPLES



feature/login

bugfix/payment-timeout

hotfix/2.0.1

release/2.4.0

chore/deps



QUICK CHECK BEFORE ANY ACTION

`git branch --show-current && git status -sb`



WORK IN PARALLEL. MERGE WITH CONFIDENCE.





7. MERGING

Combine histories.
Ship with confidence.

Merge **smartly**. Resolve cleanly.
Keep your history **intentional**.

MERGE TYPES

Fast-Forward



No merge commit. Linear history.

Three-Way Merge



Creates a merge commit.

Squash Merge



One new commit with combined changes.

THE MERGE WORKFLOW



1. Preview
`git diff`
`main...HEAD`



2. Merge
`git merge`
`feature/login`



3. Resolve (if needed)
`git status`
`git add <file>`



4. Continue
`git merge`
`--continue`



5. Push
`git push`
`origin main`

MERGE CONFLICTS

```

<<<<<<< HEAD
const port = 3000;
=====
const port = 8080;
>>>>>> feature/config
  
```



Inspect
`git diff --name-only`
`--diff-filter=U`



Resolve
Edit files and
`git add <file>`



Continue
`git merge --continue`

KEY MERGE OPTIONS

`--ff-only`

Only fast-forward merges.
Fail if a merge commit is needed.

`--no-ff`

Always create a merge commit.
Preserve the merge point.

`--squash`

Combine changes without
a merge commit.

`--abort`

Abort the merge and go back
to the pre-merge state.

`-X ours / -X theirs`

Prefer our or their changes
for eligible conflicts.

BRANCH MANAGEMENT



Create & switch
`git switch -c feature/login`



List branches
`git branch -vv`



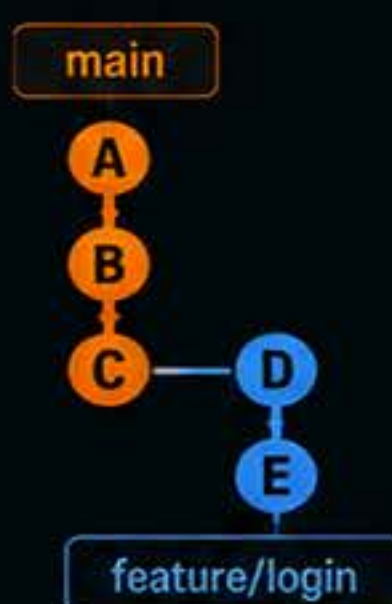
Rename branch
`git branch -m old new`



Delete branch
`git branch -d feature/login`



Publish & track
`git push -u origin feature/login`



TRACKING & UPSTREAM



Set upstream
`git push -u origin feature/login`



Show upstream
`git rev-parse --abbrev-ref @{upstream}`



Divergence
`git rev-list --left-right --count`
`HEAD...@{upstream}`



Remote branches
`git branch -r`

Example output:

2 **1**
ahead **behind**

2 commits only on
your branch

1 commit only on
upstream

MERGE ESSENTIALS

- ✓ Preview before you merge.
- ✓ Understand fast-forward vs. merge commit.
- ✓ Resolve conflicts carefully.
- ✓ Review history after merging.
- ✓ Keep history intentional and clean.



SAFETY RULES

- ⚠ Always inspect before destructive actions.
- ⚠ Do not force-delete blindly.
- ⚠ Fetch before comparing with remote.
- ⚠ Know which branch you are on.
- ⚠ Use the right merge option for the job.



MERGE CONFIDENTLY. HISTORY TELLS YOUR STORY.





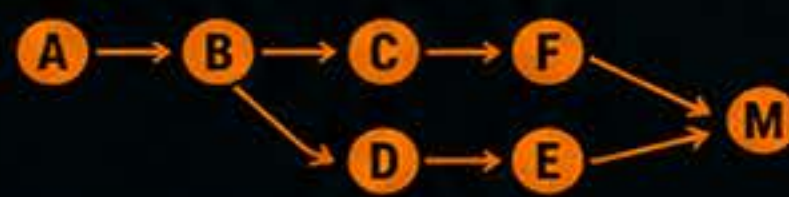
8. REBASING

Rewrite history.
Keep it clean & linear.

Rebase **moves** your commits to a new base, creating a **smooth** and easy-to-read history.

MERGE vs REBASE

Merge (preserves topology)



Results in a merge commit.

Rebase (linear history)



Replays commits on top of the new base.

REBASE WORKFLOW



INTERACTIVE REBASE COMMANDS

pick	Keep commit as-is
reword	Keep, but edit message
edit	Stop to amend/split
squash	Combine with previous, edit message
fixup	Combine with previous, drop message
drop	Remove commit
exec	Run shell command
break	Pause the rebase

CONFLICT RESOLUTION

```

<<<<<<<< HEAD
const port = 3000;
=====
const port = 8080;
>>>>>>> feature/config
  
```

- Edit the file and choose the correct content.
- Stage the resolved file.
`git add <file>`
- Continue the rebase.
`git rebase --continue`

REBASE TYPES

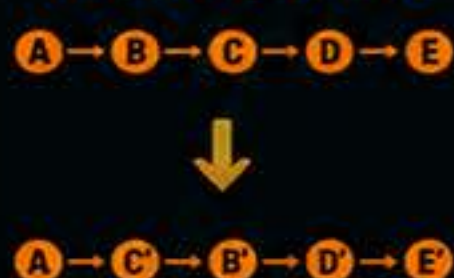
- Basic
`git rebase main`
- Interactive
`git rebase -i HEAD~5`
- Squash
`git rebase -i --autosquash`
- Preserve merges
`git rebase --rebase-merges main`

COMMON SCENARIOS

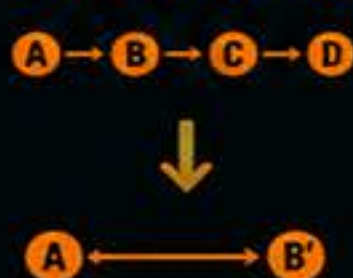
Rebase feature



Reorder commits



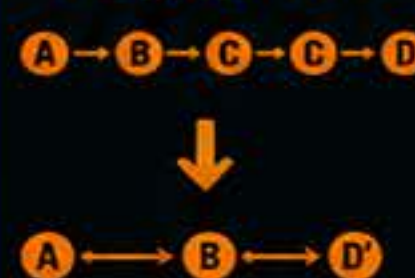
Squash commits



Split a commit



Drop commit



SAFETY RULES



- ✓ Never rebase published/shared history.
- ✓ Always fetch before rebasing.
- ✓ Inspect the history before & after.
- ✓ Resolve conflicts carefully.
- ✓ Use `--force-with-lease` when pushing.
- ✓ Have a backup branch if the history is important.

QUICK REBASE GUIDE

```

git status           # check your state
git fetch origin      # update remote refs
git switch feature/login # go to your branch
git rebase origin/main # replay on latest base
# resolve conflicts if needed
git rebase --continue # finish rebase
git push --force-with-lease # publish safely
  
```



CLEAN HISTORY. STRONG REPOSITORIES. SMARTER TEAMS.





9. REMOTE REPOSITORIES

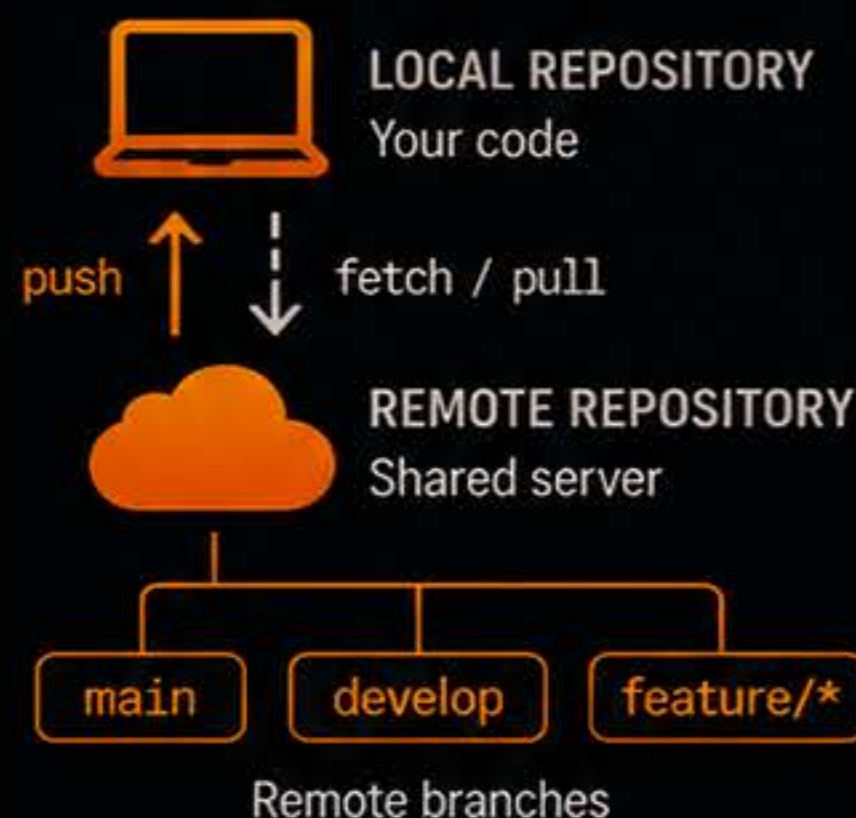
Sync. Collaborate. Ship.

Master your remotes. Keep your code in sync.



Important: origin/main is NOT your local main.
`git fetch` updates your remote-tracking branches.
`git pull` fetches and integrates.

THE BIG PICTURE



REMOTE WORKFLOW



REMOTE INSPECTION (COMMANDS)

	List remotes	<code>git remote</code>
	Show URLs	<code>git remote -v</code>
	Inspect a remote	<code>git remote show origin</code>
	See remote refs	<code>git ls-remote origin</code>
	Clean stale refs	<code>git fetch --prune</code>
	List remote branches	<code>git branch -r</code>
	Show tracking info	<code>git branch -vv</code>

COMPARE & STATUS

	What I have locally but remote doesn't	<code>git log origin/main..HEAD</code>
	What remote has that I don't	<code>git log HEAD..origin/main</code>
	Both sides (counts)	<code>git rev-list --left-right --count HEAD...origin/main</code>
	Is remote ancestor of my branch?	<code>git merge-base --is-ancestor origin/main HEAD</code>
	See what changed vs remote	<code>git diff --stat origin/main...HEAD</code>

PUSHING CHEAT SHEET

Push changes	Push new branch	Force push safely	Delete remote branch	Push tags	Dry-run push
<code>git push</code>	<code>git push -u origin feature/login</code>	<code>git push --force-with-lease</code>	<code>git push origin --delete feature/login</code>	<code>git push --tags</code>	<code>git push --dry-run</code>

WHEN TO USE WHAT?

Download only <code>git fetch origin</code>	Download + merge <code>git pull --no-rebase</code>	Download + rebase <code>git pull</code>
Safe. Updates remote-tracking branches only.	Keeps full history.	Linear history.



Pro Tip

Always fetch before big operations. Inspect. Understand. Then act.

`git fetch` → inspect → integrate → push



STAY IN SYNC. STAY IN CONTROL.

10. UNDOING CHANGES

Restore. Reset. Revert.

Use the right command at the right level.


WORKING TREE

Your files


STAGING AREA

git add


LOCAL REPOSITORY

Commits & history


REMOTE REPOSITORY

Shared with others



Important: restore, reset, and revert are **NOT** interchangeable.

restore changes files/staging state.

reset moves references and can rewrite history.

revert creates a new commit that undoes changes.

THE RIGHT TOOL FOR THE JOB


UNSTAGED CHANGES

Discard local changes in your files.

```
git restore <file>
```

Ex: `git restore app.py`


STAGED CHANGES

Unstage changes from the index.

```
git restore --staged <file>
```

Ex: `git restore --staged .`


LOCAL COMMITS (YOUR HISTORY)

Move HEAD and rewrite history.

```
--soft keep staged (mostly re-commit)
--mixed keep unstaged (working tree)
--hard discard changes (dangerous)
```

Ex: `git reset --soft HEAD~1`


PUBLIC COMMITS (SHARED HISTORY)

Create a new commit that undoes the change.

```
git revert <commit>
```

Ex: `git revert abc1234`


IN-PROGRESS OPERATIONS

Abort or continue merge/rebase.

```
git merge --abort
git rebase --abort
git rebase --continue
```

Ex: `git rebase --abort`

RESET MODES EXPLAINED

- COMMIT / HEAD
- INDEX (STAGING)
- 📄 WORKING TREE

--soft

A → B → C

Index & working tree stay the same.

Moves HEAD back.

Index & working tree stay the same.

--mixed (default)

A → B → C

Index reset. Working tree stays the same.

Moves HEAD back.

Index reset. Working tree stays the same.

--hard

A → B → C

Index & working tree reset. Changes lost.

Moves HEAD back.

Index & working tree reset. Changes lost.

REFLOG IS YOUR SAFETY NET

Every move is recorded. Recover lost work:

```
$ git reflog
abc9999 HEAD@{0}: reset: moving to HEAD-3
def8888 HEAD@{1}: rebase (finish)
cde7777 HEAD@{2}: commit: Add auth
bcd6666 HEAD@{3}: commit: Fix timeout
abc5555 HEAD@{4}: checkout: moving from main
```

Create a recovery branch:

```
$ git branch recovery abc9999
$ git switch recovery
```

MERGE (preserves topology)

A → B → C → F
D → E → M

- ✓ Keeps original branch history
- ✓ Creates a merge commit
- ✓ Easy to understand
- ✓ Good for shared history

REBASE (linear history)

A → B → C → D → D' → E' → F'

- ✓ Replays commits on top
- ✓ Linear, clean history
- ✓ Commit IDs change
- ✓ Requires force push if shared

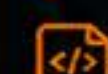
REBASE WORKFLOW



1. Update
`git fetch origin`



2. Rebase
`git rebase origin/main`



3. Resolve conflicts
`edit files & git add`



4. Continue
`git rebase --continue`



5. Push safely
`git push --force-with-lease`



SAFETY CHECKLIST

- ✓ Check your state: `git status`
- ✓ Inspect changes: `git diff / git diff --cached`
- ✓ Review history: `git log --oneline --decorate`
- ✓ Create backup: `git branch backup-before-undo`
- ✓ Use `--abort / --continue` when needed
- ✓ Force push safely: `--force-with-lease`

WHEN TO USE WHAT?

- Unstaged file change → `git restore <file>`
- Staged change → `git restore --staged <file>`
- Rewrite your local history → `git reset`
- Undo a shared commit → `git revert <commit>`
- In-progress merge → `git merge --abort`
- In-progress rebase → `git rebase --abort`



DANGEROUS COMMANDS

- ⚠ `git reset --hard` (can lose data)
- ⚠ `git restore .` (discards changes)
- ⚠ `git restore -SW .` (resets everything)
- ⚠ `git clean -fd` (removes untracked files)
- ⚠ `git push --force` (overwrites remote)



UNDO WITH CONFIDENCE. KNOW THE IMPACT. PROTECT YOUR WORK.

11. STASH

Save now. Revisit later.

Keep your work safe without committing.



Stash ≠ Commit.

Stash is temporary and not part of history.
Commits are permanent and shareable.

THE STASH MODEL



WORKING TREE

Your local changes



`git stash push`



STASH STACK

Temporary snapshots



`git stash pop / apply`



WORKING TREE

Changes restored

QUICK WINS



SAVE WORK

Stash your changes

`git stash push`



SAVE + MESSAGE

Name your stash

`git stash push -m
"WIP login"`



INCLUDE NEW FILES

Include untracked files

`git stash push -u`



INCLUDE IGNORED

Include ignored files too

`git stash push -a`



RESTORE LATER

Apply or pop your stash

`git stash apply
git stash pop`

NEWEST

STASH STACK EXAMPLE



OLDEST

stash@{0}



On feature/login
WIP login validation

stash@{1}



On main
WIP monitoring

stash@{2}



On feature/api
WIP API refactor



Stash@{0}
is always
the newest.

INSPECT YOUR STASH



`git stash list`

List all stashes



`git stash show`

Show summary



`git stash show --stat`

Show statistics



`git stash show -p`

Show full patch



`git show stash@{2}`

Show stash commit

COMMON STASH WORKFLOWS



WORK

Make changes locally



STASH

Save changes temporarily

`git stash push -m
"WIP"`



SWITCH

Change branch or pull/rebase



DO WORK

Complete other tasks



RESTORE

Bring back your changes

`git stash pop
(or apply)`



CONTINUE

Keep working where you left off

CHOOSE THE RIGHT UNDO TOOL



Uncommitted file changes



`git restore`

Discard or restore files / index



Local commits (rewrite history)



`git reset`

Move branch & rewrite local history



Shared commits (safe undo)



`git revert`

Create a new commit that undoes changes



Work in progress



`git stash`

Save & restore temporarily



DANGEROUS COMMANDS



`git reset --hard`

Can discard commits & local changes.



`git push --force`

Can overwrite remote history.
Use `--force-with-lease` instead.



`git stash clear`

Deletes ALL stashes permanently.



`git clean -fd`

Deletes untracked files & directories.
Preview first with `git clean -nd`.



SAFETY FIRST: Always check the state, inspect your changes, and create backups (branches or stashes) before doing anything destructive.



**STASH SMART.
WORK CONFIDENTLY.**

12. TAGS & RELEASES

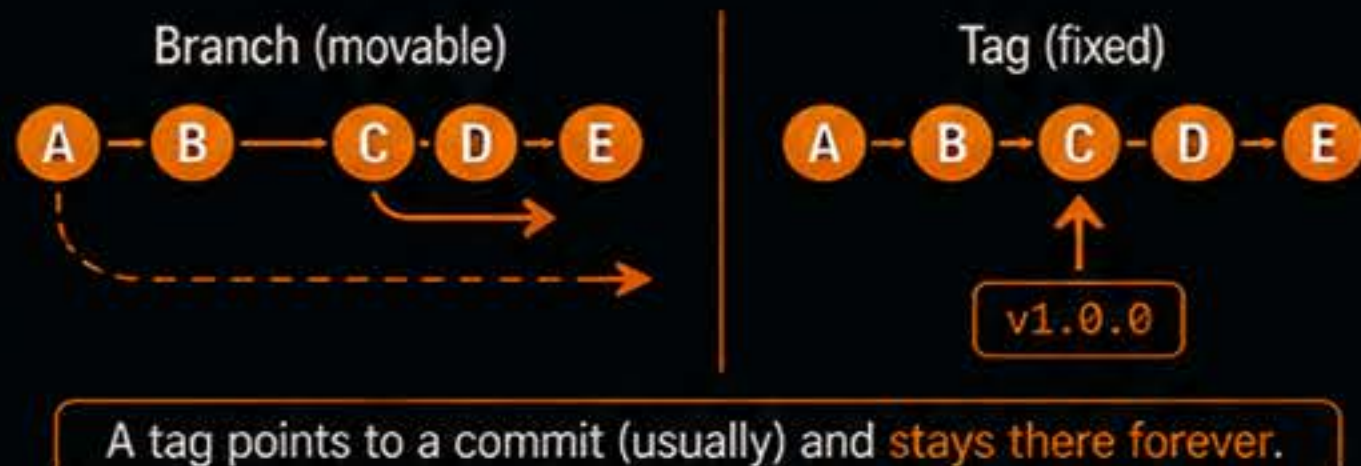
Mark important commits.
Ship with confidence.

Tags are fixed points in history —
perfect for **releases, versions & deployments**.

WHY USE TAGS?

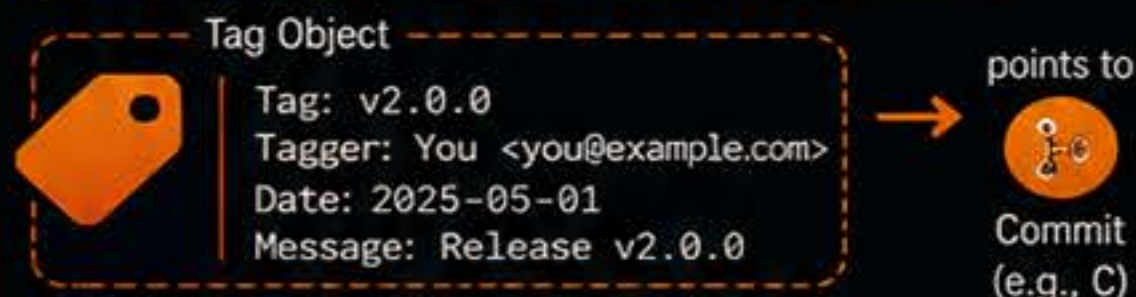
- Software releases
- Semantic versioning
- Production deployments
- Milestones & RCs
- CI/CD triggers

BRANCHES MOVE. TAGS DON'T.



ANNOTATED TAG (RECOMMENDED)

```
$ git tag -a v2.0.0 -m "Release v2.0.0"
```



Lightweight tag: `$ git tag v1.0.0`

Direct pointer to the commit (no tag object).

ESSENTIAL TAG WORKFLOW



COMMON OPERATIONS

- `$ git tag` List local tags
- `$ git tag -n` List tags with messages
- `$ git tag -l "v1.*"` Filter tags by pattern
- `$ git describe --tags --abbrev=0` Show latest version tag
- `$ git diff v1.0.0 v1.1.0` Compare releases
- `$ git log v1.0.0..v1.1.0` Show commits between releases
- `$ git push origin v1.0.0` Push one tag
- `$ git push origin --tags` Push all tags
- `$ git fetch origin --tags` Fetch tags from remote
- `$ git tag -d v1.0.0` Delete local tag
- `$ git push origin --delete v1.0.0` Delete remote tag
- `$ git tag -v v1.0.0` Verify signed tag

CREATE FROM ANY PLACE



TYPES OF TAGS

LIGHTWEIGHT TAG



ANNOTATED TAG



SIGNED TAG



STASH BEFORE CONTEXT SWITCH



SAFETY FIRST

- ✓ Always check status & diffs
- ✓ Create a backup branch for risky operations
- ✓ Understand what moves (branch, index, working tree)
- ✓ Don't force-push tags or branches casually
- ✓ Use force-with-lease instead of force
- ✓ Inspect old stashes before applying
- ✓ Don't treat stash as a replacement for commits

TAGS = IMMUTABLE RELEASE IDENTIFIERS



GOLDEN RULE

Branches change.
Tags don't.
Use tags for important milestones
that must never move.



13. SEARCHING GIT HISTORY

Find anything. Understand everything.

Powerful commands to explore **commits**, **authors**, **files**, **changes**, **bugs** & **releases** across your repo.

WHAT CAN YOU FIND?

- Commits & authors
- Changed / deleted / added code
- Specific strings & regex
- Changes to a file or range
- Merge commits & branch history
- Releases containing a commit
- The commit that changed a line

HISTORY AT A GLANCE

LINEAR LIST

```
$ git log --oneline
```

```
a1b2c3d Add authentication
b2c3d4e Fix login validation
c3d4e5f Add user model
d4e5f6a Initial project setup
```

GRAPH VIEW

```
$ git log --oneline --graph
--decorate --all
```

```
* a1b2c3d (HEAD -> main) Release v2.0.0
* * b2c3d4e Merge branch 'feature/auth'
  | * c3d4e5f Add authentication
  | * d4e5f6a Fix login validation
  |/
* e5f6a7b Update dependencies
* f6a7b8c Initial commit
```

TAGS = FIXED POINTS IN HISTORY



Tags don't move. Perfect for releases.

FIND WHAT YOU NEED



BY AUTHOR

```
$ git log --author="Alice"
```

Find commits by a person.



BY MESSAGE

```
$ git log --grep="auth"
```

Search commit messages.



BY DATE

```
$ git log --since="2 weeks ago"
```

Filter commits by date range.



BY FILE

```
$ git log -- src/login.py
```

History for a specific file.



FOLLOW RENAMES

```
$ git log --follow -- src/login.py
```

Trace file history across renames.



BY EXACT TEXT

```
$ git log -S"MAX_RETRIES"
```

Find when exact text changed.



BY REGEX

```
$ git log -G"authenticat.*"
```

Find changes matching regex.

COMPARE & INVESTIGATE

BETWEEN TAGS



```
$ git log v1.0.0..v1.1.0
$ git diff --stat v1.0.0 v1.1.0
```

See commits & changes between releases.

BRANCH DIFFERENCES



```
$ git log main..feature
$ git log --left-right main...feature
```

What's unique to each branch?

MERGE & MAINLINE



```
$ git log --merges
$ git log --first-parent
```

Find merges or follow the main integration line.

COMMON ANCESTOR



```
$ git mercur -a --contains a1b2c3d
$ git tag --contains a1b2c3d
```

Which branches or releases contain this commit?

BLAME: WHO CHANGED THIS LINE?

```
src/login.py
41 |
42 | return authenticate(user);
43 |
```

```
$ git blame -L 42,42 -- src/login.py
```

```
c3d4e5f Alice 2026-08-01 10:20
42 | return authenticate(user);
```

Perfect for debugging and code ownership.

STASH & SWITCH WITH CONFIDENCE



Unfinished work
M app.py
M config.yaml
?? debug.log

```
$ git stash push -u -m "WIP before switch"
```

Switch branches or pull/rebase

```
$ git stash pop
```

RELEASE WORKFLOW

- 1 \$ git status
- 2 \$ git switch main && git pull --ff-only
- 3 Run tests
- 4 \$ git tag -a v1.5.0 -m "Release v1.5.0"
- 5 \$ git show v1.5.0
- 6 \$ git push origin v1.5.0
- 7 Verify: \$ git ls-remote --tags origin



SAFETY RULES

- ✓ Always check status & diffs first
- ✓ Fetch before comparisons or pulls
- ✓ Inspect before destructive ops
- ✓ Avoid moving published tags
- ✓ Prefer force-with-lease over force
- ✓ Use tags as immutable release IDs
- ✓ Name stashes and tags clearly

\$ git log --oneline	Compact history	\$ git log -- path/to/file	File history	\$ git log A..B	Commits in range	\$ git fetch origin --tags	Fetch tags
\$ git log --graph --all	Graph of all history	\$ git log --follow -- file	Follow renames	\$ git diff A B	Compare versions	\$ git show-ref --tags	Show tag refs
\$ git show <commit>	Show commit details	\$ git log -S"text"	Exact text changes	\$ git tag -a v1.0.0 -m "..."	Create annotated tag	\$ git describe --tags --abbrev=0	Latest tag
\$ git log --author="Alice"	Search by author	\$ git log -G"regex"	Regex changes	\$ git push origin v1.0.0	Push one tag	\$ git stash push -u -m "WIP"	Save work
\$ git log --grep="auth"	Search commit messages	\$ git blame file	Blame file	\$ git push --tags	Push all tags	\$ git stash pop	Restore & drop
\$ git log --since="2026-08-01"	Search by date	\$ git merge-base A B	Common ancestor	\$ git ls-remote --tags origin	List remote tags	\$ git branch -a --contains <sha>	Find containing branches

Remember: git log answers "WHERE?", git show answers "WHAT?", git blame answers "WHO?", and tag!ock important moments forever.



14. COMPARING BRANCHES & COMMITS

Know exactly what changed, where, and why.

`git diff`

`git log`

`git merge-base`

`git show`

`git range-diff`

`git rev-list`



`git diff`

What changed in the content?

Content differences



`git log`

Which commits are different?

Commit differences



`git show`

What did this commit change?

Inspect a specific commit



`git merge-base`

Where did these histories last converge?

Find the common ancestor



`git range-diff`

How do two series of commits differ?

Compare commit series



`git rev-list`

Which commits are reachable from one reference but not another?

List commit differences



HEAD

Last committed snapshot



Index / Staging Area

Proposed next commit



Working Tree

Current files on disk

THREE STATES, THREE COMPARISONS

`git diff`

Working Tree

VS

Index

(Staging Area)

```
...
- line removed
+ line added
```

Unstaged changes

`git diff --cached`

Index

VS

HEAD

(Last Commit)

```
...
- 1: ==
- 1: ==
```

Staged changes

`git diff HEAD`

Working Tree + Index

VS

HEAD

(Last Commit)

```
...
- 1: ==
- 1: ==
```

All uncommitted changes

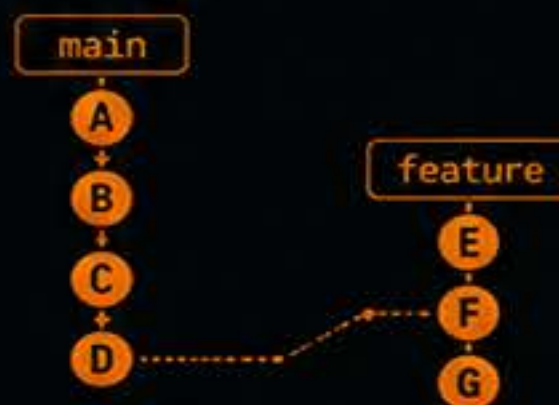
SNAPSHOT COMPARISON



`git diff main feature`

Compare D ↔ G (branch tips)

COMPARE BRANCHES THE RIGHT WAY



`git diff main...feature`

Compare C (merge-base) ↔ G (changes feature introduced)

COMMIT DIFFERENCES

`git log main..feature`

Commits on feature but not on main

`git log feature..main`

Commits on main but not on feature

`git log --left-right main...feature`

Commits unique to either side
 < = only on left
 > = only on right

POWERFUL COMPARISON OPTIONS



`--name-only`
 Show only changed files



`--name-status`
 Show file statuses (A/M/D/R/C)



`--stat`
 Show change statistics



`--word-diff`
 Word-level differences



`-w / -b`
 Ignore whitespace changes



`-M`
 Detect renames



`-C`
 Detect copies

PRACTICAL USE CASES

- ✓ Code review (what did this branch introduce?)
- ✓ Are we ahead or behind the remote?
- ✓ Which files changed?
- ✓ Which commits are unique to a branch?
- ✓ Compare releases and tags
- ✓ Review reworked commits with range-diff

MERGE BASE AT THE HEART



`git merge-base main feature`

Find the common ancestor (C) for meaningful comparisons.

AUTOMATION-FRIENDLY



Check if branches differ:

`git diff --quiet main feature`

Exit status: 0 = identical, 1 = differ

List changed files (safe for scripts):

`git diff -z --name-only main feature`



REMEMBER:

DIFFERENT COMMANDS ANSWER DIFFERENT QUESTIONS.
 Use the right tool to get the right answer.

15. CONFLICT RESOLUTION

Conflicts happen. Resolve with confidence.



Merge conflicts



Rebase conflicts



Cherry-pick conflicts



Stash conflicts



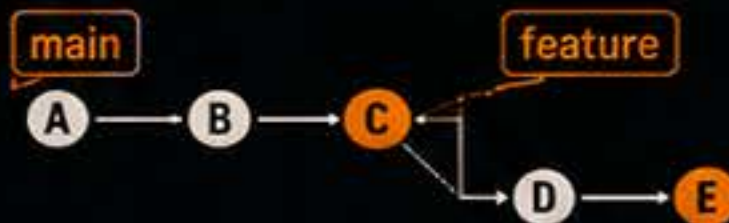
Tools & strategies



Prevent & collaborate

WHAT IS A GIT CONFLICT?

Git can't automatically decide how to combine changes from different branches, rebases, cherry-picks, or stashes.



CONFLICT (content): Merge conflict in config/app.yml

CONFLICT MARKERS

<<<<<<<< HEAD

port=8080

=====

port=9090

>>>>>>> feature

<<<<<<< HEAD

Current branch

=====

Separator

>>>>>>> feature

Incoming branch

RESOLVED EXAMPLE

```
port=8080
backup_port=9090
```

WHY CONFLICTS HAPPEN



Same line modified



File renamed differently



Delete vs modify



Binary file changes



Rebase conflicts



Cherry-pick conflicts

CONFLICT LIFECYCLE



Operation starts

merge / rebase / cherry-pick / stash



Conflict detected

Git stops and reports



Resolve conflicts

Edit files, remove markers



git add

Mark files as resolved



Continue

Complete the operation



MERGE CONFLICTS

```
$ git merge feature
# resolve conflicts...
$ git add file.js
$ git commit
```



REBASE CONFLICTS

```
$ git rebase main
# resolve conflicts...
$ git add file.js
$ git rebase --continue
$ git rebase --abort
```



CHERRY-PICK CONFLICTS

```
$ git cherry-pick abc123
# resolve conflicts...
$ git add file.js
$ git cherry-pick --continue
$ git cherry-pick --abort
```



STASH CONFLICTS

```
$ git stash pop
# resolve conflicts...
$ git add .
$ git commit
```

RESOLUTION STRATEGIES



KEEP OURS

```
git checkout --ours <file>
Keep current branch version
```



KEEP THEIRS

```
git checkout --theirs <file>
Keep incoming version
```

INSPECT CONFLICT VERSIONS

```
git ls-files -u
```

- 1 base (common ancestor)
- 2 ours (current branch)
- 3 theirs (incoming branch)

Extract content:

```
git show :1:file.txt
```

```
git show :2:file.txt
```

```
git show :3:file.txt
```

DETECTION COMMANDS

```
$ git status
```

Show conflicts

```
$ git diff
```

Show conflict markers

```
$ git diff --name-only --diff-filter=U
```

List unresolved files

MERGE TOOLS



VSCode



Meld



KDiff3



Beyond Compare



P4Merge

Launch: `git mergetool`

CONFIGURATION

Set tool:

```
git config --global
merge.tool vscode
```

Enable rerere:

```
git config --global
rerere.enabled true
```

Conflict style:

```
merge
```

(default)

```
diff3
```

(more context)

```
zdiff3
```

(even better)

COMPLETION COMMANDS

```
$ git merge --continue
```

```
$ git rebase --continue
```

```
$ git cherry-pick --continue
```

```
$ git revert --continue
```

QUICK REFERENCE



Show conflicts
`git status`



Unresolved files
`git diff --name-only --diff-filter=U`



Keep ours
`git checkout --ours <file>`



Keep theirs
`git checkout --theirs <file>`



Mark resolved
`git add <file>`



Continue
`merge/rebase/cherry-pick --continue`



Abort
`merge/rebase/cherry-pick --abort`



GUI tool
`git mergetool`



1 2 3
`git ls-files -u`



Remember resolutions
`rerere.enabled true`



CONFLICTS ARE NORMAL. GOOD TEAMS RESOLVE THEM FAST AND LEARN FROM THEM.



16. CHERRY-PICK

Pick the commit. **Keep the value.**

Apply specific commits to the current branch without merging the whole branch.



Backport fixes



Selective changes



Hotfixes



Release branches



Safe & controlled



IMPORTANT

Cherry-picking creates a **NEW COMMIT** on the current branch.

The original commit remains unchanged.



D = original commit

D' = new commit (same changes)

HOW IT WORKS



Cherry-pick applies the changes of **D** and creates **D'** on the current branch.

WHEN TO CHERRY-PICK

- ✓ Backport a bug fix to a release branch
- ✓ Move a hotfix between branches
- ✓ Apply a specific feature commit elsewhere
- ✓ Apply selected commits (not the whole branch)
- ✓ Recover a useful commit from another line

NOT A MERGE. NOT A REBASE.
JUST THE CHANGE YOU NEED.



COMMON COMMANDS



APPLY ONE COMMIT

```
git cherry-pick a1b2c3d
```

Apply a single commit.



APPLY A RANGE

```
git cherry-pick A^..B
```

Apply all commits from A to B (inclusive).



MULTIPLE COMMITS

```
git cherry-pick A B C
```

Apply several commits in the given order.



NO COMMIT

```
git cherry-pick -n a1b2c3d
```

Apply changes without committing.



EDIT MESSAGE

```
git cherry-pick -e a1b2c3d
```

Edit the commit message.



MERGE COMMIT

```
git cherry-pick -m 1 a1b2c3d
```

Cherry-pick a merge commit (choose parent).

CONFLICT WORKFLOW



★ Always test after cherry-picking, especially when backporting fixes.

CHERRY-PICK VS OTHERS



MERGE

Integrates the branch

Keeps history & all commits



REBASE

Replays your commits on another base

Rewrites history



CHERRY-PICK

Copies specific commit(s) to current branch

Creates new commits with same changes

QUICK CHEAT SHEET

```
$ git cherry-pick <commit>
```

Apply one commit

```
$ git cherry-pick <A> <B>
```

Apply multiple commits

```
$ git cherry-pick <A^..B>
```

Apply a range (inclusive)

```
$ git cherry-pick -n <commit>
```

Apply without committing

```
$ git cherry-pick -e <commit>
```

Edit commit message

```
$ git cherry-pick -m 1 <merge>
```

Cherry-pick merge commit

```
$ git cherry-pick --continue
```

Continue after conflict

```
$ git cherry-pick --skip
```

Skip current commit

```
$ git cherry-pick --abort
```

Abort operation



PRO TIP

Inspect before you pick. Understand the change, dependencies, and impact. A good cherry-pick saves time. A bad cherry-pick creates problems.



Small commits.
Clear history.
Happy teams.

REFLOG & RECOVERY

Lost commits aren't always lost.

Reflog remembers where your references have been so you can recover what you thought was gone.



IMPORTANT

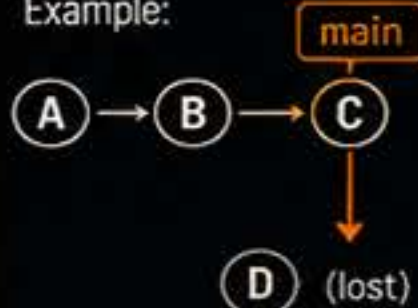
Reflog is **LOCAL** and temporary.
Not a replacement for remotes or backups.



WHAT IS REFLOG?

Tracks updates to references like HEAD and branches.

Example:



`git reflog`

```
d456789 HEAD@{0}: reset: moving to B
c345678 HEAD@{1}: commit: D
b234567 HEAD@{2}: checkout: main
a123456 HEAD@{3}: commit: C
```

Commit D is no longer reachable, but it's **still** in the reflog!

CONFLICT MARKERS

```
<<<<<<< HEAD
port=8080
=====
port=9090
>>>>>> feature
```

Resolve and remove markers:

```
port=8080
backup_port=9090
```



RECOVERY POWER



Recover deleted branches



Recover after reset --hard



Recover after rebase



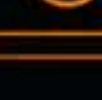
Recover after amend



Recover lost commits



Recover files



Find previous HEAD positions

COMMON RECOVERY SCENARIOS

DELETED BRANCH

`git reflog`

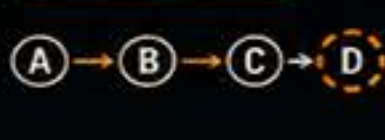
`git branch recovered HEAD@{5}`



RESET --HARD

`git reflog`

`git branch recovery HEAD@{1}`



AFTER REBASE

`git reflog show feature`

`git branch before-rebase feature@{6}`



AMENDED COMMIT

`git reflog`

`git branch original HEAD@{1}`



FORCE PUSH

`git reflog`

`git branch remote-recovery HEAD@{8}`



LOST COMMIT

`git fsck --unreachable`

`git show <id>`

`git branch saved <id>`

WHERE TO LOOK

HEAD reflog `git reflog`
Branch reflog `git reflog show <branch>`
Limit entries `git reflog -n 20`
By time `git show HEAD@("yesterday")`
Inspect commit `git show HEAD@{1}`



CONFLICT DURING CHERRY-PICK



Cherry-pick

`git cherry-pick <commit>`



Conflict

CONFLICT (content)



Resolve

Edit files
Remove markers



Stage

`git add <file>`



Continue

`git cherry-pick --continue`

Abort: `git cherry-pick --abort` | Skip: `git cherry-pick --skip`

REFLOG ENTRY FORMAT

```
e4f7a11
HEAD@{0}:
commit: Fix API endpoint
```

→ Commit / reference target
→ Reflog position
→ Operation
→ Description

CHERRY-PICK vs MERGE vs REBASE



MERGE

Integrates an entire branch



Preserves branch history & topology



REBASE

Replays your commits on another base



Rewrites commits (new IDs)



CHERRY-PICK

Copies selected commit(s) to current branch



Creates NEW commits with same changes



DANGEROUS COMMANDS

✗ `git reset --hard`
✗ `git reflog expire --expire=now --all`
✗ `git gc --prune=now`
✗ `git gc --aggressive`
✗ `git prune`
✗ `git reflog delete`

These can permanently remove your ability to recover.

OBJECTS & fsck



Find unreachable objects:
`git fsck --full --unreachable`



Store in lost-found:
`git fsck --lost-found`



Create branch from found commit:
`git branch recovered <id>`

QUICK RECOVERY WORKFLOW



STOP
Don't do more damage



Check reflog
`git reflog`



Find commit
`git show HEAD@{N}`



Create branch
`git branch recovery`



Verify
`git log recovery`



Restore / Use
reset / cherry-pick as needed



WHY REFLOG MATTERS

Git forgets.
Reflog remembers.
And **that's** how you recover.

CHERRY-PICK CHEAT SHEET

`$ git cherry-pick <commit>` Apply one commit
`$ git cherry-pick <A> <B> <C>` Apply multiple commits
`$ git cherry-pick <A>^..<B>` Apply inclusive range
`$ git cherry-pick -n <commit>` Apply without committing
`$ git cherry-pick -e <commit>` Edit commit message

`$ git cherry-pick -m 1 <merge>` Pick merge commit (mainline 1)
`$ git cherry-pick --continue` Continue after conflict
`$ git cherry-pick --skip` Skip current commit
`$ git cherry-pick --abort` Abort operation
`$ git cherry-pick -s <commit>` Add Signed-off-by

`$ git cherry-pick -S <commit>` GPG-sign the result
`$ git status` Check conflict state
`$ git diff --name-only --diff-filter=U` Unresolved files
`$ git reflog` Show HEAD history
`$ git fsck --lost-found` Find lost objects

GIT BISECT

Find the commit that introduced the bug, fast. Git does the **binary search** for you.



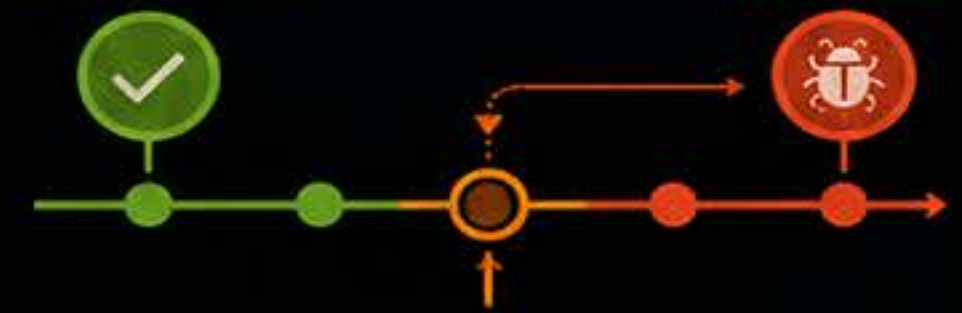
Important: Cherry-pick creates a new commit. The original commit remains unchanged.

HOW IT WORKS

KNOWN GOOD



KNOWN BAD



Git picks the middle commit.
You test it → **good** or **bad** → repeat.

WHY USE IT?



Backport bug fixes



Find regressions



Build or test failures



Performance drops



Behavior changes



Long histories = save time

BASIC WORKFLOW



`git bisect start`
Start session



`git bisect bad`
Mark known bad
(current commit)



`git bisect good`
Mark known good
(older commit)



Test candidate
Run tests



`git bisect good`
Works



`git bisect bad`
Broken



First bad commit
found

`git bisect reset`
Return to normal
state

CONFLICTS DURING CHERRY-PICK



Conflict

Git stops
the operation



Resolve files

Edit manually
or use mergetool



Stage

`git add <file>`



Continue

`git cherry-pick`
`--continue`



Skip

`git cherry-pick`
`--skip`



Abort

`git cherry-pick`
`--abort`

KEY COMMANDS

```
git bisect start
git bisect bad [<commit>]
git bisect good [<commit>]
git bisect skip
git bisect status
git bisect log
git bisect run <cmd>
git bisect reset
```

SIMPLE HISTORY



Tell Git good or bad.
It cuts the search space in half each time.

RESULT

First bad commit:



(Introduced the bug)

Inspect it:
`git show E`

AUTOMATED BISECT

```
$ git bisect run ./test.sh
Running ./test.sh
# test fails
# test passes
# test fails
...
```

Git runs the test **automatically**
and finds the culprit for you.

RECOVERY WITH REFLOG



Deleted branch?
See where it was.



Reset by mistake?
Find previous HEAD.



Rebase went wrong?
Go back.



Amended the wrong commit?
Recover the original.



Force pushed?
Find the old commits.

REFLOG EXAMPLE

```
e4f7a11 HEAD@{0}: reset: moving to HEAD~1
8c92b30 HEAD@{1}: commit: Fix API endpoint
5d31a44 HEAD@{2}: commit: Add API
a123456 HEAD@{3}: checkout: moving from
feature to main
b234567 HEAD@{4}: rebase (finish)
```

Use HEAD@{N} or time expressions:

```
git show HEAD@{1}
git show HEAD@{"yesterday"}
```

RECOVERY OPTIONS



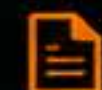
Create a recovery branch
`git branch recovered HEAD@{1}`



Move back in time
`git reset --hard HEAD@{1}`
(use carefully!)



Inspect without moving
`git switch --detach HEAD@{1}`



Recover a file
`git restore --source=HEAD@{1} -- file.js`



Apply one commit
`git cherry-pick HEAD@{1}`

GOOD vs BAD



- Bug is **NOT** present
- Tests pass
- Bug **IS** present
- Tests fail

AUTOMATION EXIT CODES

- | | |
|------|-------------------|
| 0 | Good |
| 1 | Bad |
| 125 | Skip (untestable) |
| 126 | Cannot execute |
| 127 | Command not found |
| 128+ | Error |



RULE OF THUMB

- Use reliable tests.
- Keep tests deterministic.
- Bisect small logical ranges.
- Skip bad commits.
- Always `git bisect reset` when done.



TIME SAVER

- Linear search:
check **N** commits
- Bisect search:
 $\sim \log_2(N)$ commits

★ **Remember:** Reflog remembers. Bisect finds. Together, they save your commits—and your day.

19. GIT SUBMODULES

Include another Git repository as a dependency while keeping projects independent.



Important: The parent repository records a **specific commit** of the submodule. It does **NOT** track the latest automatically.

WHAT IS A SUBMODULE?

Parent Repository

Submodule Repository



points to



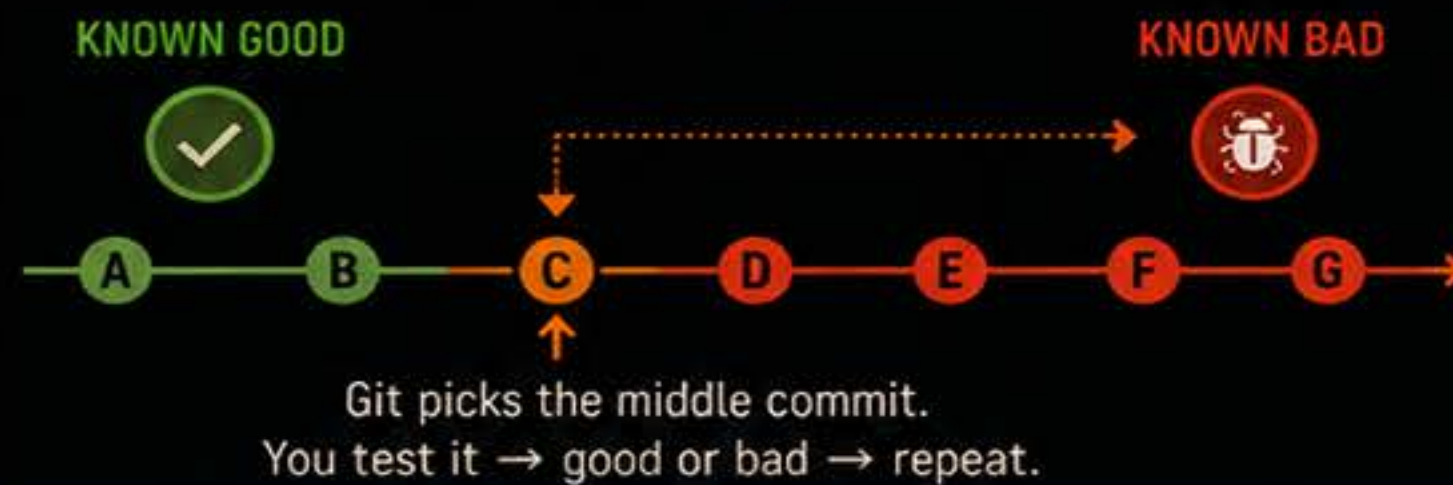
Specific Commit

The parent stores refs, not file contents.

WHY USE SUBMODULES?

- Independent dependency
- Pin exact commit
- Different release cycle
- Share across projects
- Reproducible builds

HOW IT WORKS (BINARY SEARCH FOR BUGS)



Binary search reduces work from N checks to $\sim \log_2(N)$.

KEY FILES



.gitmodules

Stores configuration:

```
[submodule "libs/library"]
  path = libs/library
  url = https://example.com/library.git
  branch = main (optional)
```



Gitlink in index/commit

160000 commit abc123...

Represents the submodule commit.

CORE WORKFLOW



Repeat until Git identifies the commit that introduced the bug.

ADD A SUBMODULE



```
git submodule add
<URL> <PATH>
```

- Create .gitmodules
- Create gitlink entry
- Commit & push

CLONE WITH SUBMODULES



```
git clone --recurse-submodules <URL>
```

Or after cloning:

```
git submodule update --init --recursive
```

UPDATE SUBMODULES



```
git submodule update --init --recursive
```

To remote branches:

```
git submodule update --remote --recursive
```

CHECK STATUS



```
git submodule status
--recursive
```

- not initialized
+ differs from recorded
U merge conflict

UPDATE TO NEW COMMIT

- `cd <submodule>`
- `git switch main`
- `git pull`
- `git m'`
- `cd ../..`
- `git add <submodule>`
- `git commit -m "Update submodule"`
- `git push`

CONFLICTS?



Resolve as usual:

- Edit files
- `git add <file>`
- `git commit`

Check: `git status`

MERGE TOOLS



```
git mergetool
```

Tools: VSCode • Meld
KDiff3 • Beyond Compare
P4Merge

USE OURS / THEIRS

```
git checkout --ours <file>
Keep current branch
```

```
git checkout --theirs <file>
Keep incoming change
```

REMOVE SUBMODULE



```
git submodule deinit -f -- <PATH>
git rm -f <PATH>
git commit -m "Remove submodule"
```

RECURSIVE CONFIG

```
git config submodule.recurse true
git config push.recurseSubmodules
on-demand
git config fetch.recurseSubmodules
true
```

BISECT CHEAT SHEET

<code>git bisect start</code>	Start bisect session
<code>git bisect bad</code>	Mark current commit bad
<code>git bisect good <c></code>	Mark commit good
<code>git bisect skip</code>	Skip current commit
<code>git bisect status</code>	Show current state
<code>git bisect log</code>	Show decisions
<code>git bisect run <cmd></code>	Run tests automatically
<code>git bisect replay <log></code>	Replay previous session
<code>git bisect reset</code>	End session & return

RECOVERY WITH REFLOG

- `git reflog` Find previous HEAD positions
- `git show HEAD@{N}` Inspect candidate commit
- `git branch recovery %E@{N}` Create safe name reference
- Verify: `git log --oneline recovery`
- Option A: `git reset --hard HEAD@{N}`
- Option B: `git cherry-pick HEAD@{N}`
- Finally: `git bisect reset` (if in bisect)



Reflog remembers. Your lost work is often still there — recover it before it's gone!



COMMON PROBLEMS & FIXES

- Empty submodule directory?
→ `git submodule update --init --recursive`
- Submodule URL changed?
→ `git submodule sync --recursive`
- Parent can't push?
→ `git config push.recurseSubmodules on-demand`
- Detached HEAD in submodule?
→ Normal! Use `git switch main` if you want a branch
- Nested submodules missing?
→ `git submodule update --init --recursive`



Submodule = separate repo



Parent stores commits, not files



Always commit submodule updates in parent



Use --recursive for nested repos



Test deterministically for git bisect run



Reflog & bisect save your day



20. GIT WORKTREES

Multiple working directories, **one** repository.

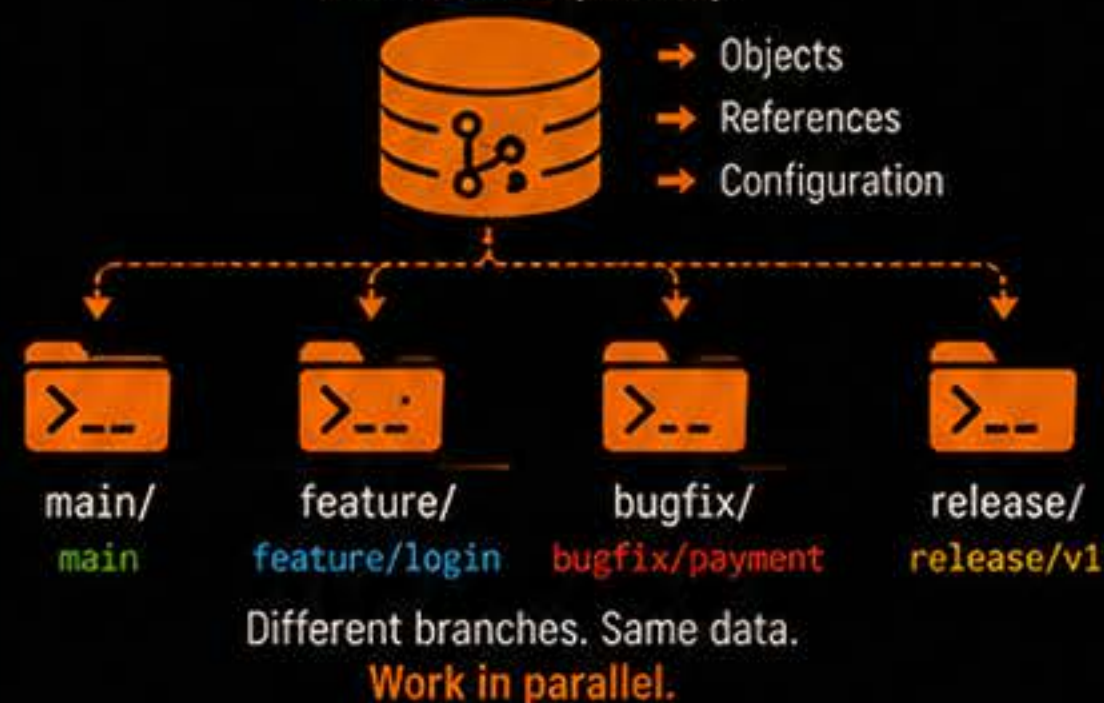
Work on **many** branches at the same time.



Important: Worktrees share the same Git **object** database. Each worktree is linked to a specific branch or commit.

WORKTREE ARCHITECTURE

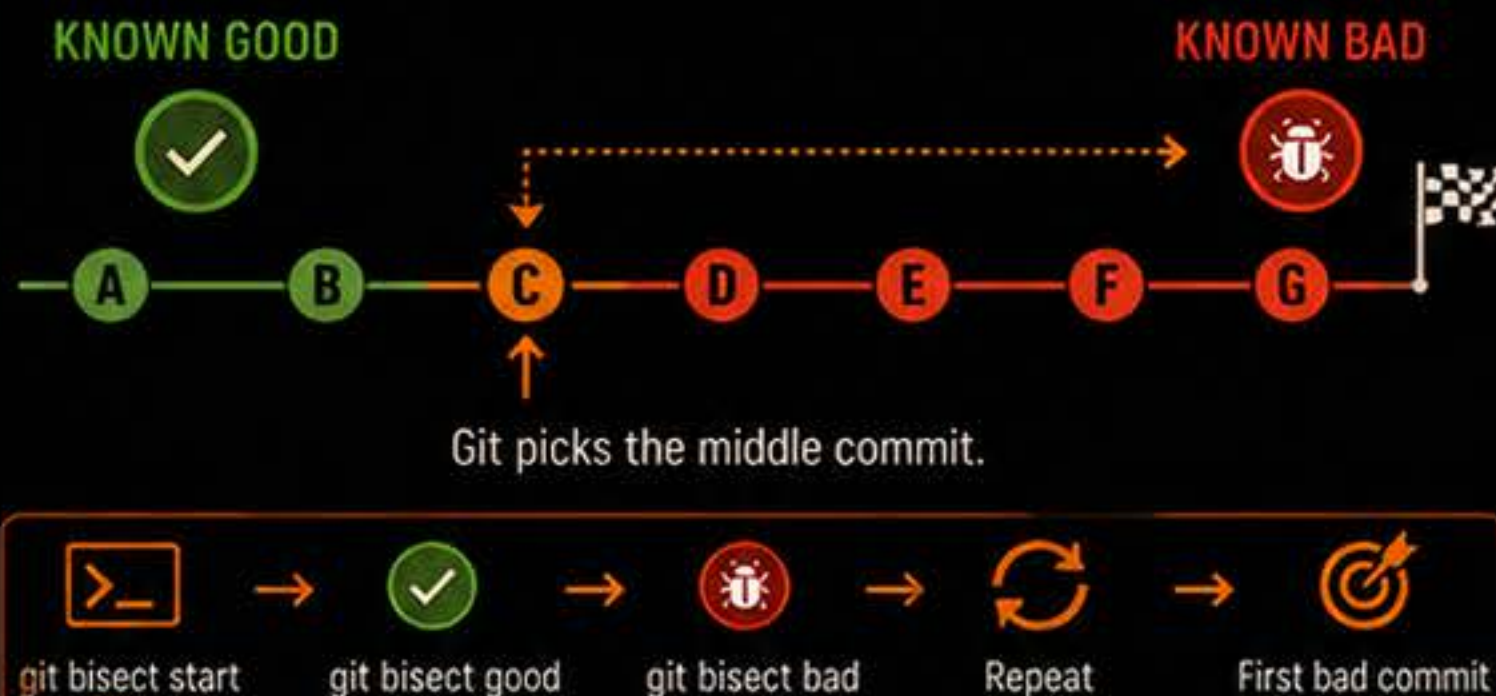
Shared Git Repository



WHY USE WORKTREES?

- Parallel development
- Hotfixes without stashing
- Release branches side by side
- Code reviews & comparisons
- Run tests or builds for multiple branches
- Independent, isolated workspaces
- Faster than cloning multiple repos

HOW IT WORKS (BINARY SEARCH FOR BUGS)



COMPLEX HISTORY?

Git automatically narrows the range until the first bad commit is found.

CREATE WORKTREES



Existing branch

```
$ git worktree add
../feature-login
feature/login
```



New branch

```
$ git worktree add -b
feature/search
../feature-search
```



Specific commit

```
$ git worktree add --detach
../review
a1b2c3d
```

Multiple directories, **one** repository.
No need to switch branches.

ESSENTIAL WORKTREE COMMANDS



List

```
$ git worktree list
...
```



Verbose

```
$ git worktree list
--verbose
```



Status

```
$ git worktree status
.....
```



Prune

```
$ git worktree prune
--dry-run
```



Move

```
$ git worktree move
OLD NEW
```



Lock

```
$ git worktree lock
../feature
```



Unlock

```
$ git worktree unlock
../feature
```



Remove

```
$ git worktree remove
../feature
```

COMMON WORKFLOWS



Feature Development

1. worktree add -b
2. develop
3. commit & push



Hotfix

1. worktree add -b
2. fix & test
3. push urgently



Code Review

1. worktree add --detach
2. review & test
3. remove



Release Testing

1. worktree add --detach
2. build & test
3. compare versions

BISECT (DEBUG REGRESSIONS)



KEY FILE & CONFIG

.gitmodules

```
[submodule "libs/library"]
path = libs/library
url = https://github.com/example/library.git
branch = main (optional)
```



Useful configs

```
$ git config submodule.recurse true
$ git config fetch.recurseSubmodules true
$ git config push.recurseSubmodules on-demand
```



TROUBLESHOOTING

- Submodule empty? → `git submodule update --init --recursive`
- URL changed? → `git submodule sync --recursive`
- Stale worktree entry? → `git worktree prune`
- Detached HEAD? → normal for pinned commits
- Parent won't push? → push submodule commit first
- Need more details? → `git diff --submodule=log`



GOLDEN RULES

Prefer worktrees over stashing.

- ✓ Always commit submodule updates in parent.
- ✓ Keep submodule commits intentional and reviewable.
- ✓ Use `--recursive` for nested submodules.
- ✓ Do not delete worktree directories manually.
- ✓ Verify with: `git worktree list --verbose`
- ✓ Stay safe, stay productive!



Work in parallel. Stay organized. Ship faster.

One repo. Many workspaces.



21. GIT LFS

Store big files smartly.

Git keeps **pointers**, LFS stores the **data**.



.psd



.zip



Video



Audio



Images



Assets

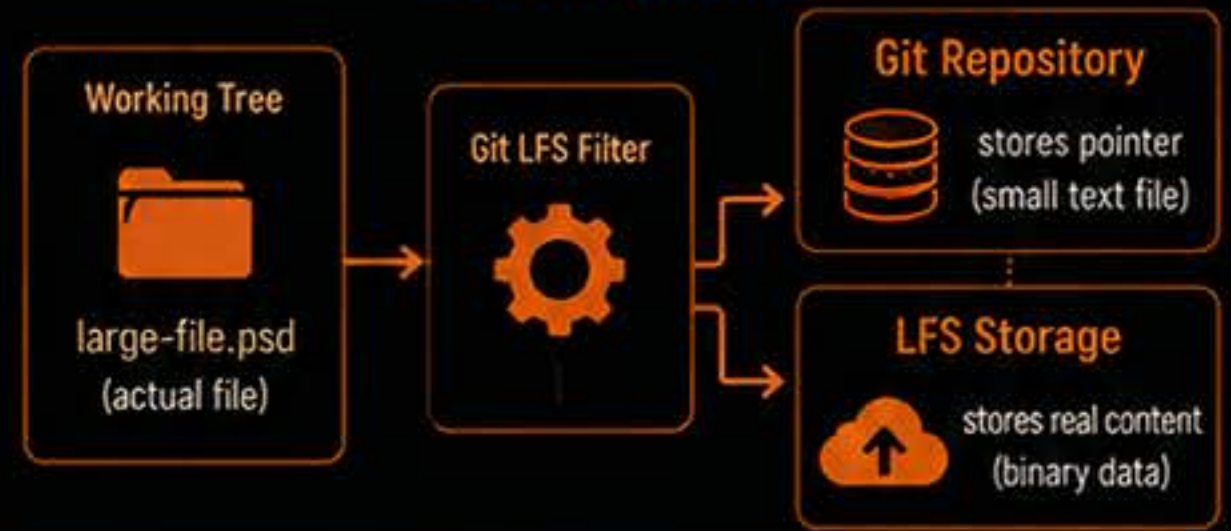


Datasets



Binaries

HOW IT WORKS



Fast clones. Small repos. Full files when you need them.

WHY USE GIT LFS?

- ✓ Keep repositories small & fast
- ✓ Handle large binaries efficiently
- ✓ Pin exact file versions
- ✓ Better for teams & CI/CD
- ✓ Avoid huge history bloat

LFS POINTER FILE (IN GIT REPO)

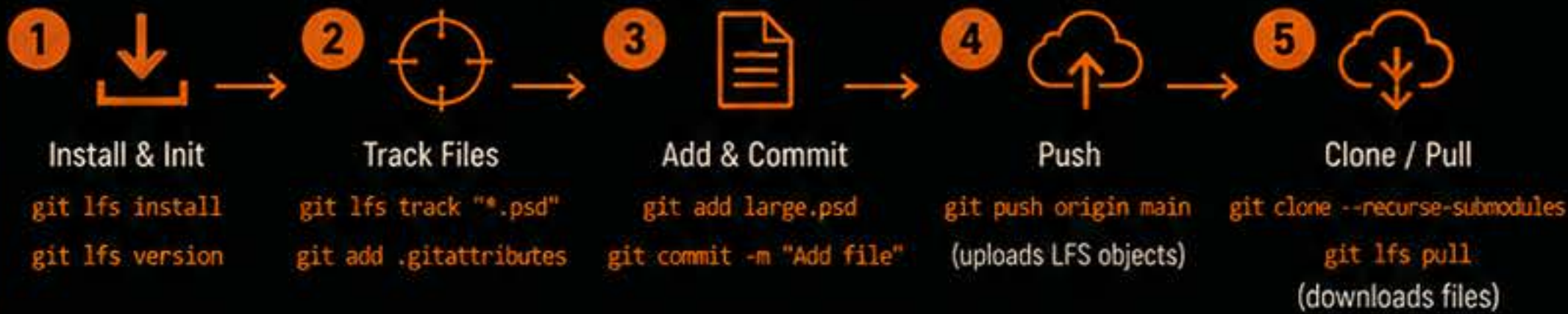
```
version https://git-lfs.github.com/spec/v1
oid sha256:8f8c2f...a3b9d1e
size 524288000
```

This tiny file is committed.
The real content lives in LFS storage.

LFS ARCHITECTURE



CORE WORKFLOW

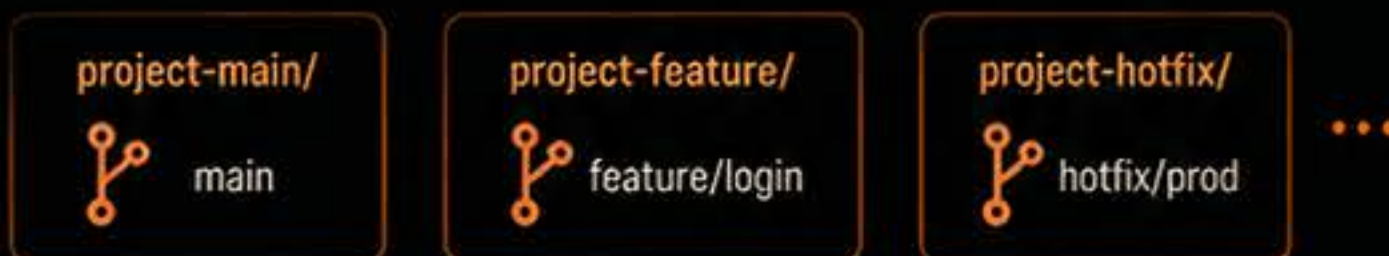


CHECK STATUS



```
git lfs ls-files
git lfs ls-files --long
git lfs env
git lfs logs last
```

CREATE WORKTREES & USE IN PARALLEL



Multiple branches. No constant switching.

MIGRATE EXISTING FILES TO LFS



Rewrites history. Coordinate before migrating in shared repos!

IMPORTANT COMMANDS

<code>git lfs install</code>	Initialize Git LFS	<code>git lfs migrate import</code>	Migrate history to LFS
<code>git lfs track "pattern"</code>	Track files with LFS	<code>git lfs env</code>	Show LFS environment
<code>git lfs ls-files</code>	List LFS managed files	<code>git lfs logs last</code>	Show last LFS log
<code>git lfs fetch</code>	Fetch LFS objects	<code>git check-attr filter -- file</code>	Check LFS tracking
<code>git lfs pull</code>	Download LFS content	<code>git lfs prune</code>	Prune local LFS cache
<code>git lfs push origin main</code>	Upload LFS objects	<code>git lfs reinstall</code>	Reinstall hooks & filters
<code>git lfs untrack "pattern"</code>	Stop tracking pattern		

GREAT FOR



Videos



Audio



Images



Assets



Datasets



Binaries



CAD Files



Docs & More

GOLDEN RULES



Track only large files.



Commit `.gitattributes`.



Don't modify LFS storage manually.



Use `lfs pull` to get real files.



Avoid history rewrites in shared repos.



Check logs when things go wrong.



Use worktrees for parallel work.

★ Small repo. Fast operations. Big files managed the right way.





22. GIT HOOKS



Automate. Enforce. Supercharge your Git workflow.

Run custom scripts at **key points** in the **Git lifecycle**.

WHAT ARE GIT HOOKS?



Git Command



Git Lifecycle
Event



Hook Script



Exit 0 → Continue



Non-zero → Abort

WHAT CAN HOOKS DO?



Validation



Code
Formatting



Linting



Tests



Commit Policy
Enforcement



Branch Policy
Enforcement



Release
Automation



CI/CD
Integration



Deployment
Automation



Repository
Administration

TWO MAIN CATEGORIES



CLIENT-SIDE HOOKS
Run on the developer's
machine.



SERVER-SIDE HOOKS
Run on the Git server.

POPULAR HOOKS (EXAMPLES)



pre-commit

Before commit
Run checks, tests,
linting, formatting.



commit-msg

Validate commit
message format
and rules.



pre-push

Run tests before
pushing to remote.



post-merge

Update dependencies,
regenerate files,
prepare environment.



pre-receive

Server: validate
push before
accepting.



post-receive

Server: trigger CI,
deployments,
notifications.

...

HOOK WORKFLOW (EXAMPLE)

Commit Lifecycle



pre-commit



prepare-commit-msg



commit-msg



post-commit

Push Lifecycle



pre-push



Objects
Transferred



HOOK BEST PRACTICES

- ✓ Keep hooks fast and developer-friendly
- ✓ Validate staged files, not the whole repo
- ✓ Use hooks for convenience, CI for enforcement
- ✓ Never store secrets in hooks
- ✓ Make hooks portable and secure
- ✓ Provide clear and helpful error messages
- ✓ Review hooks before enabling
- ✓ Enforce critical rules on the server



Git hooks turn Git into a **powerful automation engine**.
Use them wisely. Build better workflows. **Ship with confidence.**

Read the full chapter
in the Git LFS section!





23. REPOSITORY MAINTENANCE

Keep your Git repository **fast**, **healthy** and ready for the **long run**.



WHY MAINTENANCE MATTERS



Efficient



Compact



Consistent



Fast



Diagnosable



Safe to
Archive



Safe to
Transfer



Healthy
Over Time

WHAT HAPPENS DURING MAINTENANCE?



Maintenance optimizes these structures for **speed**, **size** and **reliability**.

KEY MAINTENANCE COMMANDS

<code>> _</code>	
<code>git maintenance run</code>	Run scheduled maintenance
<code>git gc</code>	Garbage collect repository
<code>git prune</code>	Remove unreachable objects
<code>git repack</code>	Reorganize packfiles
<code>git pack-refs --all</code>	Pack references
<code>git count-objects -vH</code>	Inspect repository size
<code>git fsck --full</code>	Verify repository integrity
<code>git commit-graph write</code>	Speed up commit operations
<code>git multi-pack-index write</code>	Optimize multi-pack lookup
<code>git rerere gc</code>	Clean old conflict resolutions

POPULAR MAINTENANCE TASKS



`gc`

Clean up and
repack



`prune`

Remove unreachable
objects



`repack`

Consolidate
packfiles



`pack-refs`

Pack reference
files



`reflog expire`

Expire old
reflog entries



`commit-graph`

Speed up commit
traversal



`multi-pack-index`

Efficient lookup
across packs

RECOMMENDED WORKFLOW



1. INSPECT
`count-objects`
`fsck --full`



2. VERIFY
Check integrity
and reflogs



3. MAINTAIN
Run maintenance
or `gc`



4. VERIFY AGAIN
Confirm health
and size



Measure, verify, maintain, repeat.

Small maintenance today = big **performance** tomorrow.

DON'T FORGET

- ✓ Back up before destructive maintenance
- ✓ Never prune before checking reflogs
- ✓ Keep recovery windows after history rewrites
- ✓ Use scheduled maintenance for large repos
- ✓ CI/CD benefits from `commit-graph` & MIDX
- ✓ Maintenance changes storage, not history!



A healthy repository is a **fast** repository.
Maintain it, **trust** it, **scale** it.



Read the full chapter in the
Git Reference on GitHub! →



24. REPOSITORY DIAGNOSTICS

Inspect. Understand. Troubleshoot.
Know your repository **inside out**.



WHAT DIAGNOSTICS TELL YOU



Branch &
history state



Staged,
unstaged,
untracked



Repository
health &
integrity



Remotes &
synchronization



Configuration
& environment



Size &
object storage



Why something
behaves
unexpectedly

START HERE

1



`git status -sb`
Quick view of branch
and working tree

2



`git branch -vv`
Upstream tracking,
ahead/behind info

3



`git log --oneline --graph --decorate --all`
Visualize your history

4



`git fsck --full`
Check repository integrity

5



`git count-objects -vH`
Inspect object storage & size

DIAGNOSTIC POWER TOOLS



`git rev-parse`
Resolve refs &
repository paths



`git show-ref`
Inspect all
references



`git merge-base`
Find common
ancestors



`git rev-list`
Count & analyze
commits



`git diff`
Inspect changes
(worktree/index/HEAD)



`git ls-files`
List tracked,
staged & untracked



`git ls-tree`
Inspect commit
tree contents



`git cat-file`
Inspect Git
objects



`git remote show`
Remote tracking
details



`git ls-remote`
Check actual remote
references



`git config`
View configuration
& origins



`git --version`
Know your
Git version

COMMON DIAGNOSES



Detached HEAD?
`git symbolic-ref --short HEAD`



Diverged branches?
`git rev-list --left-right --count HEAD...@{upstream}`



Missing commits?
`git reflog --all & git log --all`



Possible corruption?
`git fsck --full & inspect packfiles`



Repository too large?
`git count-objects -vH & du -sh .git`

DIAGNOSTIC WORKFLOW



Observe
What is the symptom?



Identify state
status, branch, rev-parse



Inspect refs & history
branch, log, show-ref, merge-base



Inspect objects
fsck, cat-file, count-objects



Inspect configuration & env
config, remotes, worktrees, submodules



Act with confidence
Only after understanding the cause

CHEAT SHEET

```
git status -sb
git branch -vv
git rev-parse --short HEAD
git rev-parse --show-toplevel
git show-ref
git log --oneline --graph --decorate --all
git merge-base main feature
git rev-list --left-right --count HEAD...@{upstream}
git diff / git diff --cached / git diff HEAD
git ls-files / git ls-tree -r --name-only HEAD
git cat-file -t <obj> / git cat-file -p <obj>
git fsck --full
git count-objects -vH
git verify-pack -v .git/objects/pack/*.idx
git remote -v / git remote show origin
git ls-remote origin
git config --list --show-origin --show-scope
git rev-parse --is-shallow-repository
git worktree list
git submodule status --recursive
git --version
```



DIAGNOSE BEFORE YOU ACT.

Observe → Inspect → Understand → Then Change.
Good diagnostics save time, prevent data loss, and
lead to confident solutions.



Diagnostics turn confusion into clarity.
Know your repository. Master your workflow.



25. GIT OBJECTS / INTERNALS

Git stores **everything** as **objects**.
Understand the **building blocks**.



THE 4 CORE OBJECT TYPES



BLOB

File contents



TREE

Directory snapshot
(points to blobs/trees)



COMMIT

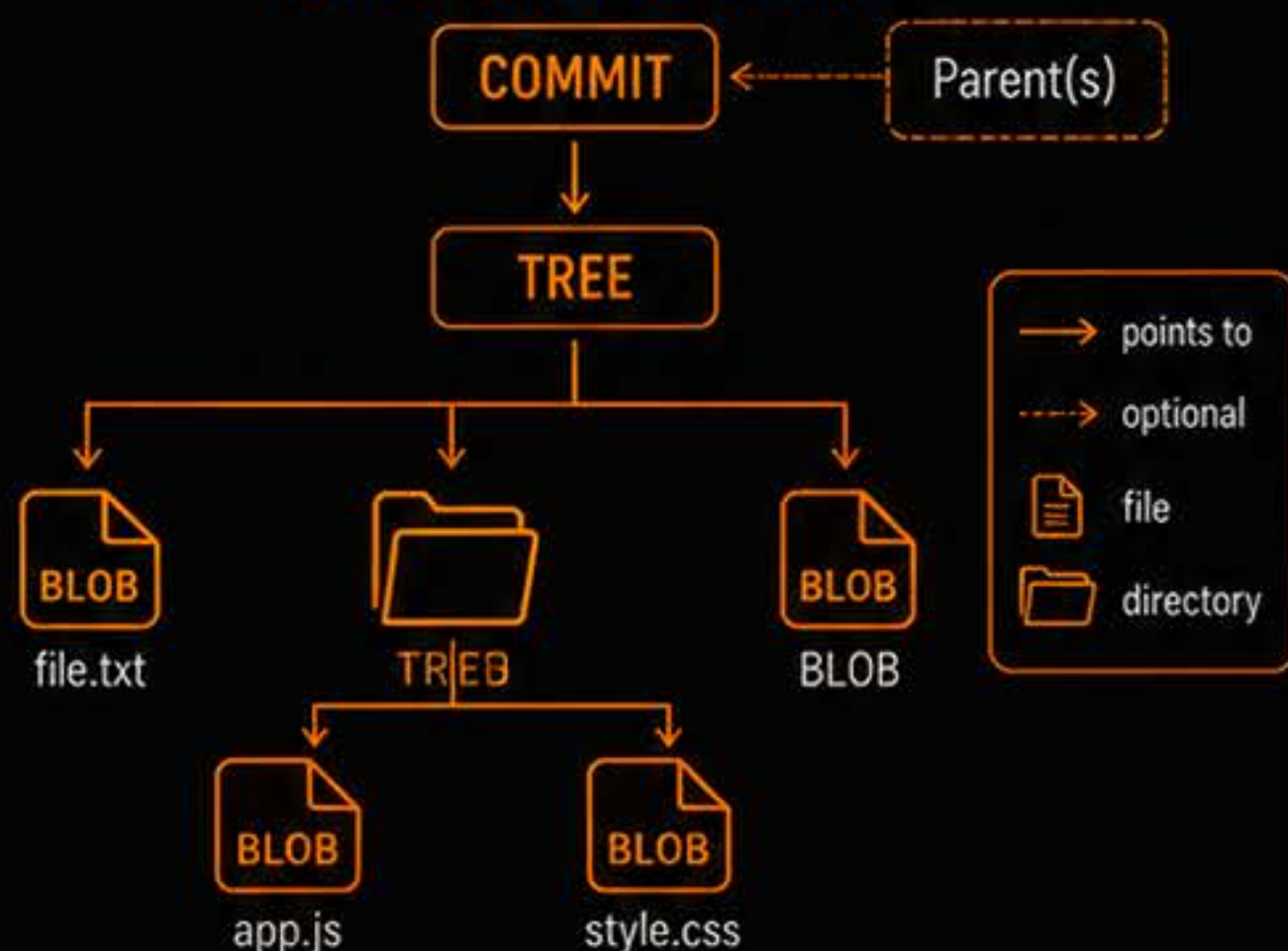
History, tree, parents,
author, message



TAG

Annotated reference
with metadata

HOW OBJECTS CONNECT



KEY INTERNAL CONCEPTS



Content-addressable

Object ID is based on content.



Branches = References

A branch name points to a commit.



Tags

Lightweight tags point directly.
Annotated tags are objects.



Reachability

Objects reachable from refs are kept.
Others may be pruned.



Immutability

Objects never change.
New objects are created.

REPOSITORY STRUCTURE (.GIT)



Objects stored here: loose objects or packfiles

ESSENTIAL INSPECTION COMMANDS



<code>git cat-file -t <obj></code>	Show object type
<code>git cat-file -p <obj></code>	Inspect object content
<code>git ls-tree <commit></code>	List tree contents
<code>git rev-parse <ref></code>	Resolve reference
<code>git show-ref</code>	List references
<code>git fsck --full</code>	Check object database
<code>git count-objects -vH</code>	Repository statistics

OBJECT DATABASE WORKFLOW



WHY IT MATTERS

- ✓ Debug problems
- ✓ Recover lost commits
- ✓ Understand history
- ✓ Inspect packed objects
- ✓ Build Git tools
- ✓ CI/CD automation
- ✓ Repository maintenance
- ✓ Git hosting systems



Git is more than commands — it's an **object database**.
Know the internals. **Solve anything.**



Read the full chapter in the
Git Reference on GitHub! →



26. IGNORING FILES



Tell Git what **NOT** to track.

Cleaner repo. **Safer** commits. Happier devs.

WHAT .GITIGNORE DOES

It hides untracked files and directories from Git.



COMMON IGNORE RULES

	Directory	node_modules/ dist/
	Specific file	.env config.local.json
	Extension	*.log *.tmp
	Anything	*.local
	Anywhere (recursive)	**/cache/
	Negation (re-include)	*.json !config.example.json



IMPORTANT: .gitignore does **NOT** remove files already tracked by Git.

You must remove them from the index first.

DIAGNOSE IGNORES

	<code>git check-ignore file</code>	Is this file ignored?
	<code>git check-ignore -v file</code>	Which rule is ignoring it?
	<code>git status --ignored</code>	Show ignored files
	<code>git ls-files --ignored</code>	List ignored files
	<code>git ls-files -- file</code>	Is it already tracked?

WHERE IGNORE RULES COME FROM

	.gitignore	→		Shared with everyone. Commit it.
	.git/info/exclude	→		Local to this repo. Don't commit it.
	core.excludesFile (Global)	→		Personal rules across all repositories.

TRACKED vs IGNORED

	≠		≠	
TRACKED		IGNORED		UNTRACKED
Git tracks it. Ignored rules don't apply.		Untracked AND matched by an ignore rule.		Not tracked and not ignored.

COMMON ACTIONS



IGNORE SMART, COMMIT SAFE

- ✓ Ignore build artifacts, dependencies, logs, and secrets.
- ✓ Use specific rules — avoid ignoring too much.
- ✓ Never rely on .gitignore for secret protection.
- ✓ Check rules with `git check-ignore -v`.
- ✓ Keep .gitignore readable and documented.
- ✓ Review before running `git clean -fdX`.



QUICK CHEAT SHEET

<code>git status --ignored</code>	Show ignored files
<code>git check-ignore -v file</code>	Why is it ignored?
<code>git ls-files -- file</code>	Is it tracked?
<code>git rm --cached file</code>	Stop tracking (keep local)
<code>git add -f file</code>	Force-add ignored file
<code>git clean -ndX</code>	Preview clean ignored
<code>git clean -fdX</code>	Remove ignored files



A **clean .gitignore** leads to a clean repo.
Ignore the noise. Focus on **what matters**.



Read the full chapter in the
Git Reference on GitHub! →



27. FILE TRACKING

Control what Git **tracks**, **stages**, and **commits**.
Master your files. **Ship** better code.



THE 3 AREAS OF GIT

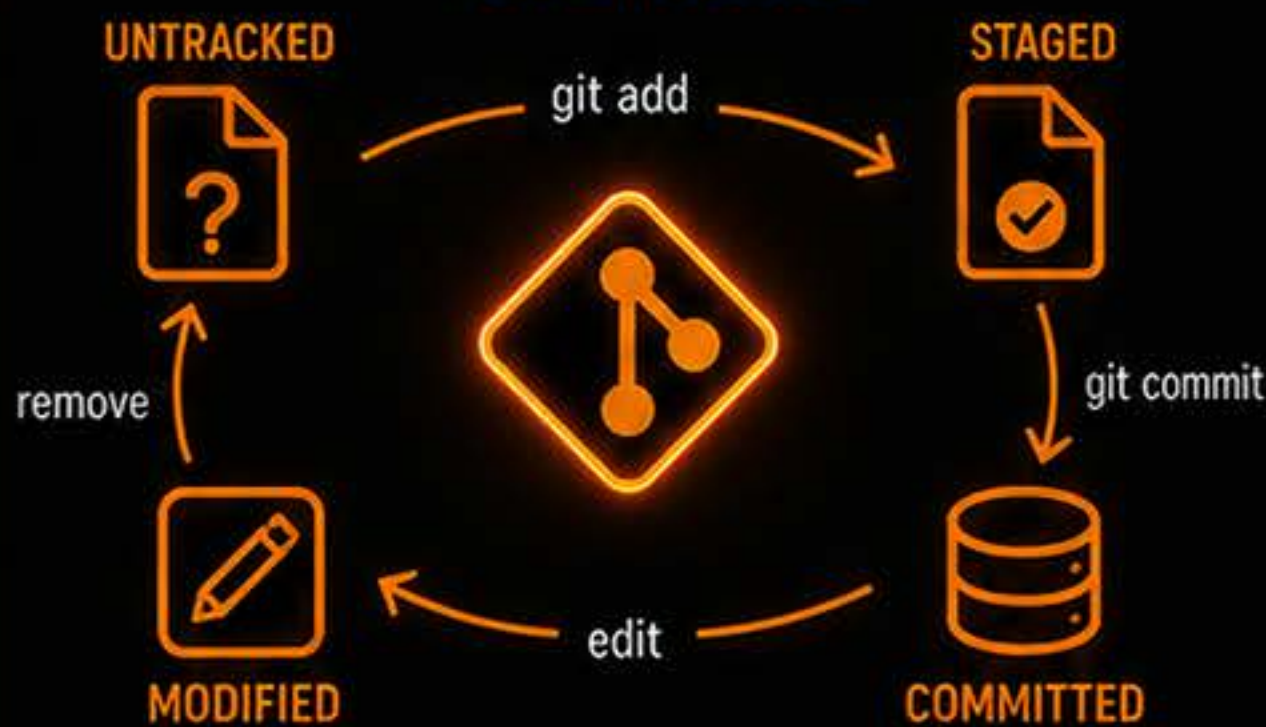


File states: **Untracked** • **Tracked** • **Modified** • **Staged** • **Deleted**
Renamed • **Copied** • **Ignored**

FILE STATUS AT A GLANCE

??	Untracked	D	Deleted (staged)
A	Added (staged)	D	Deleted (unstaged)
M	Modified (unstaged)	R	Renamed (staged)
M	Modified (staged)	C	Copied (staged)
MM	Modified (staged + more unstaged)	!!	Ignored

FILE LIFECYCLE



! **.gitignore** hides files from Git.
It does **NOT** stop tracking files already tracked.

HIGH-VALUE COMMANDS

<code>>_</code>	<code>git status -sb</code>	See current state (branch + changes)
<code>📄</code>	<code>git add -A</code>	Stage all changes
<code>📁</code>	<code>git add -p</code>	Stage changes interactively
<code>↶</code>	<code>git restore --staged <file></code>	Unstage a file
<code>🕒</code>	<code>git restore <file></code>	Discard unstaged changes
<code>📄</code>	<code>git rm <file></code>	Delete file and stage removal
<code>🕒</code>	<code>git rm --cached <file></code>	Stop tracking but keep locally
<code>📁</code>	<code>git mv <old> <new></code>	Move or rename a file
<code>🔗</code>	<code>git ls-files</code>	List tracked files (index)
<code>📄</code>	<code>git diff --cached</code>	See what is staged
<code>📄</code>	<code>git diff</code>	See unstaged changes
<code>📄</code>	<code>git check-ignore -v <file></code>	Why is this file ignored?
<code>🗑️</code>	<code>git clean -ndX</code>	Preview ignored files to remove
<code>🗑️</code>	<code>git clean -fdX</code>	Remove ignored files (careful!)

TRACKED vs IGNORED

✓ TRACKED	!! IGNORED
Known to Git.	Ignored files are untracked and hidden from normal status.
Changes are tracked, commits include them.	

IGNORING 101 (.gitignore)

<code>📁</code> Directory	<code>node_modules/</code>
<code>📄</code> File	<code>.env</code>
<code>📁</code> Extension	<code>*.log</code>
<code>📄</code> Any file	<code>*.tmp</code>
<code>🔗</code> Recursive dir	<code>**/cache/</code>
<code>📄</code> Root relative	<code>/build/</code>
<code>🔗</code> Negation	<code>*.json</code> <code>!config.example.json</code>
<code>👤</code> Comment	<code># ignore logs</code>

MULTIPLE SOURCES OF IGNORE RULES

<code>👤</code> <code>.gitignore</code>	Shared with team. Commit it.
<code>👤</code> <code>.git/info/exclude</code>	Local to this repo. Don't commit it.
<code>🌐</code> <code>core.excludesFile</code> (Global)	Personal rules across all repositories.

DEBUG IGNORE ISSUES

1. Check status — `git status --short --ignored`
2. Is it tracked? — `git ls-files -- <file>`
3. Why ignored? — `git check-ignore -v -- <file>`
4. Review rules — `cat .gitignore`
5. Local excludes — `cat .git/info/exclude`
6. Global excludes — `git config --show-origin --get core.excludesFile`



Right files in the right place.
Stage **intentionally**. Commit with **confidence**.



Read the full chapter in the
Git Reference on GitHub! →

28. SPARSE CHECKOUT



Work with what you need.
Ignore the rest. **Same history.** Less clutter.

.gitignore



WHAT IS SPARSE CHECKOUT?



THE 3 GIT AREAS



CONE MODE (RECOMMENDED)



COMMON COMMANDS

```
>_
git sparse-checkout set --cone <dir>
git sparse-checkout add --cone <dir>
git sparse-checkout list
git sparse-checkout reapply
git sparse-checkout disable
git clone --sparse <url>
git clone --no-checkout <url>
```

PATTERN MODES



CHECK & DEBUG

```
git sparse-checkout list → Show selected paths
git status --ignored → See ignored files
git check-ignore -v <file> → Why a file is ignored
cat .git/info/sparse-checkout → Inspect rules
git config core.sparseCheckout → Is it enabled?
git clone --sparse <url>
```



IMPORTANT: `.gitignore` does **NOT** untrack files already committed.
Use `git rm --cached <file>` to stop tracking.



Ignored ≠ Untracked ≠ Tracked
Three different states. Know the difference.

FILE STATES OVERVIEW

??	Untracked	Not known to Git
A	Added (staged)	Ready for commit
M	Modified (unstaged)	Changed in working tree
M	Modified (staged)	Changed in index
MM	Modified (staged + more)	More changes not staged
D	Deleted (staged)	Will be deleted in commit
D	Deleted (unstaged)	Missing in working tree
R	Renamed (staged)	Detected as rename
C	Copied (staged)	Detected as copy
!!	Ignored	Ignored by <code>.gitignore</code> rules

GIT FILE LIFECYCLE



BEST PRACTICES

- ✓ Stage intentionally, commit confidently.
- ✓ Use `.gitignore` for noise, not for secrets already committed.
- ✓ Prefer cone mode for directory selections.
- ✓ Reapply rules after complex operations.
- ✓ Combine with partial clone for huge repos.
- ✓ Review before `git clean -fdX` ✖ (destructive!).
- ✓ Know what's tracked, staged, and ignored.
- ✓ Use `git check-ignore -v` to debug.
- ✓ Keep working tree lean & focused.



Smarter tracking. Cleaner workspace. Faster Git.
Focus on what matters — ship better code.



Read the full chapter in the
Git Reference on GitHub! →

29. GIT ARCHIVES

Export a **snapshot** of files from any Git reference into **.tar**, **.tar.gz**, or **.zip** — **without the .git** history.



WHAT IS GIT ARCHIVE?



3 AREAS OF GIT



ARCHIVE FROM ANY REFERENCE

- HEAD** (current commit)
- Branch** (e.g., main)
- Tag** (e.g., v1.2.0)
- Commit** (e.g., abc1234)

COMMON FORMATS

TAR	TAR.GZ	ZIP
- Portable - No compression	- Compressed - Widely used	- Windows-friendly - Easy to share
<code>git archive --format=tar -o source.tar HEAD</code>	<code>git archive --format=tar.gz -o source.tar.gz HEAD</code>	<code>git archive --format=zip -o source.zip HEAD</code>

CONTROL WHAT GOES IN

- Specific directory**
`git archive HEAD backend/`
- Specific files**
`git archive HEAD README.md LICENSE`
- Archive prefix**
`--prefix=project-1.0.0/`
- Exclude with .gitattributes**
`path export-ignore`

CHECK & INSPECT

<code>git sparse-checkout list</code>	Show selected paths
<code>git status --ignored</code>	See ignored files
<code>git check-ignore -v <file></code>	Why a file is ignored?
<code>tar -tzf file.tar.gz</code>	List TAR.GZ contents
<code>unzip -l file.zip</code>	List ZIP contents



ENABLE SPARSE CHECKOUT



ADD / REMOVE PATHS

Add: `git sparse-checkout add --cone docs`
 Set: `git sparse-checkout set --cone backend docs`
 Remove: `git sparse-checkout set --cone backend`
 Reapply: `git sparse-checkout reapply`
 Disable: `git sparse-checkout disable`

FILE STATES (SHORT STATUS)

?? Untracked	D Deleted (staged)
A Added (staged)	D Deleted (unstaged)
M Modified (unstaged)	R Renamed (staged)
M Modified (staged)	C Copied (staged)
MM Modified (staged + more)	!! Ignored

IMPORTANT REMINDER



.gitignore does **NOT** untrack files already committed.
Use `git rm --cached <file>` to stop tracking.

HIGH-VALUE COMMANDS

<code>>_</code>	<code>git archive -o file.tar HEAD</code>	Archive HEAD to TAR
	<code>git archive -o file.tar.gz <ref></code>	Archive ref to TAR.GZ
	<code>git archive -o file.zip <ref></code>	Archive ref to ZIP
	<code>git archive --prefix=name/ <ref></code>	Add directory prefix
	<code>git archive <ref> path/</code>	Archive a directory
	<code>git archive <ref> file1 file2</code>	Archive specific files
	<code>git archive --remote=<url> HEAD</code>	Remote archive (if supported)
	<code>git archive --list</code>	List supported formats



Share source. Ship clean.
No **.git**, just code.



Reproducible
releases



CI/CD
artifacts



Easy to
distribute



Read the full chapter in the
Git Reference on GitHub! →

30. ENVIRONMENT & AUTOMATION

Make Git predictable, scriptable and non-interactive.
Prefer **explicit, machine-readable** commands.



THE 3 AREAS OF GIT



KEY ENVIRONMENT VARIABLES

```

GIT_AUTHOR_NAME / EMAIL
GIT_COMMITTER_NAME / EMAIL
GIT_DIR, GIT_WORK_TREE
GIT_INDEX_FILE
GIT_TERMINAL_PROMPT=0
GIT_CONFIG_GLOBAL / SYSTEM
GIT_CEILING_DIRECTORIES
GIT_TRACE, GIT_TRACE2
  
```

CONFIG PRECEDENCE



HIGH-VALUE COMMANDS

<code>git rev-parse --show-toplevel</code>	Find repository root
<code>git branch --show-current</code>	Get current branch
<code>git rev-parse HEAD</code>	Get current commit (full SHA)
<code>git rev-parse --short HEAD</code>	Get short commit
<code>git status --porcelain=v2</code>	Machine-readable status
<code>git diff --quiet</code>	Check unstaged changes
<code>git diff --cached --quiet</code>	Check staged changes
<code>git diff --check</code>	Check whitespace errors
<code>git for-each-ref --format=..</code>	List refs (machine-readable)
<code>git ls-files</code>	List tracked files
<code>git ls-tree -r --name-only HEAD</code>	List files in a commit
<code>git merge-base --is-ancestor A B</code>	Check ancestry
<code>git rev-list --count HEAD</code>	Count commits
<code>git cat-file -e <object></code>	Check object exists

NON-INTERACTIVE & EXIT CODES



Disable prompts in automation:

```

GIT_TERMINAL_PROMPT=0
git fetch
  
```



Use exit codes, not text parsing.

```

if git diff --quiet; then
  echo "No changes"
else
  echo "Changes exist"
fi
  
```

COMMON EXIT CODES

0	Success / true
1	General failure / false
128	Fatal error (invalid repo, etc.)
>1	Command-specific errors

ARCHIVES (SOURCE SNAPSHOTS)



```

git archive \
  --format=tar \
  --output=project.tar HEAD
  
```

```

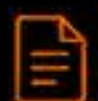
git archive \
  --format=tar.gz \
  --prefix=project-1.0.0/ \
  --output=project-1.0.0.tar.gz \
  v1.0.0
  
```

```

git archive \
  --format=zip \
  --output=project.zip HEAD
  
```



No .git directory



Use .gitattributes
export-ignore



Pipe to stdout
if needed

SPARSE CHECKOUT (PARTIAL WORKING TREE)



```

git sparse-checkout set --cone backend  Select paths
git sparse-checkout add --cone docs     Add paths
git sparse-checkout list                 Show paths
git sparse-checkout reapply              Reapply rules
git sparse-checkout disable              Return full tree
  
```



Repository (full)

```

backend/
frontend/
mobile/
docs/
infra/
  
```



Sparse working tree

```

backend/  ✓
docs/     ✓
infra/    ✗
frontend/ ✗
mobile/   ✗
  
```



IDENTITY FOR AUTOMATION

Temporary identity (no global change):

```

git -c user.name="CI Bot" \
  -c user.email="ci@example.com" \
  commit -m "Automated update"
  
```



CI/CD CHECKOUT PATTERN

```

git fetch --prune origin
git rev-parse --verify "${CI_COMMIT_SHA}^(commit)"
git checkout --detach "${CI_COMMIT_SHA}"
git status --porcelain=v2
  
```



TRACE & DIAGNOSTICS

```

GIT_TRACE=1 git status      # basic trace
GIT_TRACE2=1 git fetch      # Trace2
GIT_TRACE2_PERF=/tmp/trace.log git status
  
```



WORK WITH REFS SAFELY

```

git rev-parse --verify REF
git show-ref --verify --quiet refs/heads/main
git for-each-ref --format='% (refname:short)
  %(objectname:short)' refs/heads/
  
```



CHECK FILES IN SCRIPTS

```

git ls-files --error-unmatch file  # is tracked?
git diff --name-only               # unstaged
git diff --cached --name-only      # staged
git diff --name-only -z | xargs -0 # safe names
  
```



GOLDEN RULE

Use explicit refs, machine-readable output, exit codes, and non-interactive settings.
Git automation = predictability + safety.



Automate with confidence. Build with precision. Ship with Git.



31. COMMON DEVOPS / CI COMMANDS

Scriptable. Predictable. Non-interactive.

Build → Test → Release → Deploy



1. IDENTIFY



- Repo root
- Branch
- Commit
- Version
- Author

```
git rev-parse --show-toplevel
git branch --show-current
git rev-parse HEAD
```

2. VALIDATE



- Repository
- Clean tree
- Whitespace
- References
- Integrity

```
git status --porcelain=v2
git diff --check
git fsck --full
```

3. COMPARE



- Changes
- Between refs
- Ancestry
- Merge base
- Commit count

```
git diff --name-only A..B
git merge-base --is-ancestor A B
git rev-list --count A..B
```

4. AUTOMATE



- Non-interactive
- Exact checkout
- CI identity
- Archives
- Push / Tag

```
GIT_TERMINAL_PROMPT=0
git checkout --detach $SHA
git archive --format=tar.gz ...
```



CI/CD CHECKOUT PATTERN (REPRODUCIBLE)

```
#!/usr/bin/env bash
set -euo pipefail

git fetch --prune origin
git rev-parse --verify "${CI_COMMIT_SHA}^{commit}"
git checkout --detach "${CI_COMMIT_SHA}"
test "$(git rev-parse HEAD)" = "${CI_COMMIT_SHA}"
test -z "$(git status --porcelain)"
git diff --check
echo "CI checkout validated ✓"
```

COMMON WORKFLOWS



Fetch
--prune
--tags



Resolve
exact
commit



Validate
status
fsck



Build & Test
make
pytest



Release
archive
push tag



HIGH-VALUE COMMANDS

<code>git rev-parse --show-toplevel</code>	Repository root
<code>git branch --show-current</code>	Current branch
<code>git rev-parse HEAD</code>	Current commit (full SHA)
<code>git rev-parse --short HEAD</code>	Short commit
<code>git status --porcelain=v2</code>	Machine-readable status
<code>git diff --check</code>	Whitespace errors
<code>git diff --name-only</code>	Changed files
<code>git merge-base --is-ancestor A B</code>	Check ancestry
<code>git merge-base A B</code>	Find merge base
<code>git rev-list --count A..B</code>	Count commits
<code>git ls-remote origin</code>	List remote refs
<code>git checkout --detach SHA</code>	Exact CI checkout
<code>git archive --format=tar.gz ...</code>	Create release archive
<code>git worktree add --detach ...</code>	Isolated build workspace

GIT ENVIRONMENT VARIABLES

<code>\$ env grep ^GIT_</code>	
<code>GIT_AUTHOR_NAME / EMAIL</code>	Set author identity
<code>GIT_COMMITTER_NAME / EMAIL</code>	Set committer identity
<code>GIT_DIR</code>	Location of .git directory
<code>GIT_WORK_TREE</code>	Location of working tree
<code>GIT_INDEX_FILE</code>	Use alternative index
<code>GIT_CEILING_DIRECTORIES</code>	Limit repo discovery
<code>GIT_CONFIG_NOSYSTEM</code>	Ignore system config
<code>GIT_CONFIG_GLOBAL</code>	Override global config
<code>GIT_CONFIG_SYSTEM</code>	Override system config
<code>GIT_TERMINAL_PROMPT=0</code>	Disable interactive prompts
<code>GIT_SSH_COMMAND</code>	Control SSH behavior
<code>GIT_TRACE / GIT_TRACE2</code>	Diagnostics / tracing



COMMON ARCHIVE PATTERN

```
VERSION="$(git describe --tags --always)"
git archive \
  --format=tar.gz \
  --prefix="project-$VERSION/" \
  --output="project-$VERSION.tar.gz" \
  HEAD
sha256sum "project-$VERSION.tar.gz"
```



EXIT CODES (IMPORTANT)

0	Success
1	General failure / differences
128	Fatal (invalid repo, bad object, etc.)
>1	Command-specific errors



NEVER DO THIS IN CI

<code>git pull</code>	✗
<code>git push --force</code>	✗
Password in URL	✗
Rely on interactive input	✗
Parse human output	✗



BEST PRACTICES

- ✓ Use explicit refs
- ✓ Machine-readable output
- ✓ Check exit codes
- ✓ Non-interactive only
- ✓ Exact commit builds
- ✓ Keep secrets out
- ✓ Deterministic artifacts
- ✓ Fail fast, log clearly



AUTOMATE WITH CONFIDENCE. BUILD WITH PRECISION. SHIP WITH GIT.





32. COMMON DEVELOPER WORKFLOWS

Practical Git workflows for everyday development.



PREDICTABLE



TERMINAL FRIENDLY



TEAM READY



SAFE BY DEFAULT



GIT HOSTING COMPATIBLE



ADAPTABLE & FLEXIBLE



IMPORTANT: Commands assume “origin” is the primary remote and “main” is the primary integration branch unless stated otherwise.



WHAT'S INSIDE?

- ✓ 100+ real world workflows
- ✓ From setup to advanced workflows
- ✓ Branching, rebasing, merging
- ✓ Conflicts, recovery & safety
- ✓ Tags, releases & deployments
- ✓ Forks, upstream & collaboration
- ✓ Worktrees, stash & more

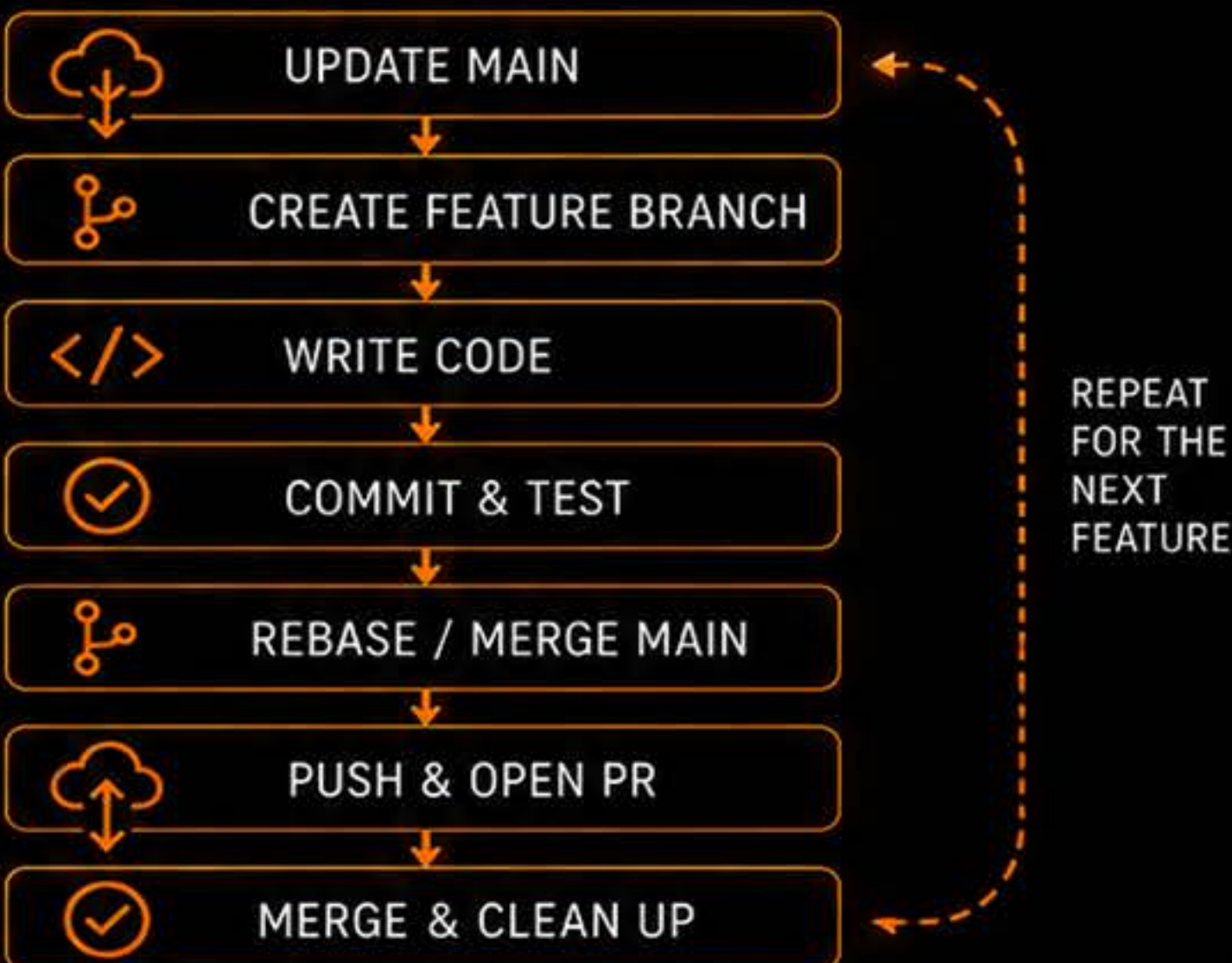


PERFECT FOR

Developers, Engineers,
DevOps & Teams



THE DEV WORKFLOW



**MASTER GIT WORKFLOWS.
SHIP BETTER. TOGETHER.**



READ THE FULL CHAPTER
AND LEVEL UP YOUR GIT GAME!



33. HIGH-VALUE COMMANDS

The Git commands that cover 90% of real-world developer workflows.



DAILY
ESSENTIALS



BRANCHING &
SYNCHRONIZATION



HISTORY &
RECOVERY



RELEASES &
DEPLOYMENTS



SAFE, FAST &
EFFECTIVE

THE DAILY DEV CYCLE



TOP COMMAND CATEGORIES

	BRANCHING	switch, branch, merge, rebase
	CHANGES	add, commit, diff, restore, reset
	REMOTE	fetch, pull, push, remote
	HISTORY	log, show, blame, reflog, bisect
	RELEASES	tag, cherry-pick, archive
	PRODUCTIVITY	stash, worktree, clean, grep

20 COMMANDS TO MEMORIZE FIRST

- | | |
|------------------|--------------------|
| 01 git status | 11 git push |
| 02 git add | 12 git merge |
| 03 git commit | 13 git rebase |
| 04 git diff | 14 git restore |
| 05 git log | 15 git reset |
| 06 git switch | 16 git revert |
| 07 git switch -c | 17 git stash |
| 08 git branch | 18 git cherry-pick |
| 09 git fetch | 19 git reflog |
| 10 git pull | 20 git tag |

WORKFLOW HIGHLIGHTS

- ✓ Start features from an updated main
- ✓ Keep commits small, focused & meaningful
- ✓ Rebase for clean history (use carefully)
- ✓ Use --force-with-lease when pushing after rebase
- ✓ Use revert for published history
- ✓ Use reflog when everything goes wrong

THE GIT MENTAL MODEL



**MASTER THESE COMMANDS.
SHIP BETTER. TOGETHER.**

Read the full chapter and level up your Git game!

34. DANGEROUS COMMANDS

Powerful. Risky. Potentially destructive.
Know what you're doing—**always**.



SAFETY RULE:

Before executing a destructive command:



git status



git log --oneline
--graph --decorate
--all



Inspect commits
and files



Understand exactly
what will change



COMMANDS THAT CAN DESTROY DATA

- ▶ `git reset --hard` Discards commits & local changes
- ▶ `git clean -fd(x)` Deletes untracked (and ignored) files
- ▶ `git push --force` Overwrites remote history
- ▶ `git push --mirror` Syncs & deletes all remote refs
- ▶ `git branch -D` Deletes branch without safety check
- ▶ `git stash clear` Removes all stashes
- ▶ `git filter-repo` Rewrites repository history
- ▶ `git reflog expire` Removes recovery references
- ▶ `git gc --prune=now` Can permanently remove objects
- ▶ `git rebase -i` Rewrites commits (new IDs)
- ▶ `git commit --amend` Replaces last commit
- ▶ `git restore --source` Overwrites files from another commit



These commands are not "bad"—but they can cause permanent loss if used carelessly.



BEFORE YOU RUN ANY DESTRUCTIVE COMMAND

- ✓ Run git status
- ✓ Check current branch
- ✓ Review recent history
- ✓ Inspect affected files
- ✓ Check remote & fetch
- ✓ Identify uncommitted work
- ✓ Preview deletions
- ✓ (git clean -nd)
- ✓ Create a backup branch
- ✓ Confirm the command
- ✓ Only then—proceed

QUICK INSPECTION SET

```
git status -sb
git branch --show-current
git remote -v
git log --oneline --graph
--decorate --all -20
git diff
git diff --cached
```



**WHEN IN DOUBT
STOP.**

**INSPECT. UNDERSTAND.
THEN DECIDE.**

SAFER ALTERNATIVES



`git reset --hard`



`git reset --soft`
or inspect first



`git clean -fd(x)`



Use `git clean -nd(x)`
first



`git push --force`



Use
`--force-with-lease`



`git branch -D`



Use
`git branch -d`



`git stash clear`



Drop individual
stashes



rewrite shared
history



Use `git revert`
instead



EMERGENCY RECOVERY

- 1 **STOP.** Do not run more commands
- 2 `git status`
- 3 `git reflog`
- 4 `git log --oneline --graph --decorate --all`
- 5 `git show <commit>`
- 6 Create recovery branch
`git switch -c recovery <commit>`
- 7 Verify and continue carefully



DANGEROUS COMBINATIONS

- ⚠ `reset --hard + push --force` Rewrites & deletes history
- ⚠ `clean -fd + reset --hard` Destroys tracked & untracked
- ⚠ `rebase -i + push --force` Rewrites many commits
- ⚠ `reflog expire + gc --prune=now` Removes recovery paths
- ⚠ `push --mirror` Can delete remote refs
- ⚠ `filter-repo + force push` Massive history rewrite



**THINK BEFORE YOU TYPE.
YOUR FUTURE SELF WILL THANK YOU.**

★ 10 GOLDEN RULES ★



1

Inspect
before
destroying



2

Preview
before
deleting



3

Prefer
revert over
rewriting



4

Prefer
`--force-with-lease`
over `--force`



5

Reflog is
powerful but
not forever



6

Untracked
files can be
lost forever



7

Never rewrite
shared
history



8

Backup before
major
rewrites



9

Rotate secrets,
then clean
history



10

Stop after a
mistake, then
recover



POWERFUL DOESN'T MEAN RISK-FREE. KNOW THE RISKS. MASTER THE SAFETY.



END

@go_evangelist



35. PRACTICAL GIT ALIASES

Shorter commands. Smarter workflows.
More productivity—**every day**.



SHORTER
COMMANDS



FASTER
WORKFLOWS



EASY TO
REMEMBER



CONSISTENT
& SAFE



TEAM
FRIENDLY



DEV & DEVOPS
READY

★ HIGH-VALUE ALIASES (EXAMPLES)

st	<code>git status -sb</code>	Quick status
lg	<code>git log --oneline --graph --decorate --all</code>	Visual history
br	<code>git branch -vv</code>	Branch overview
current	<code>git branch --show-current</code>	Current branch
aa	<code>git add --all</code>	Stage everything
ap	<code>git add --patch</code>	Interactive staging
df	<code>git diff</code>	Unstaged changes
dc	<code>git diff --cached</code>	Staged changes
fa	<code>git fetch --all --prune</code>	Fetch & prune
plr	<code>git pull --rebase</code>	Pull with rebase
psu	<code>git push -u origin HEAD</code>	Push current branch
sl	<code>git stash list</code>	List stashes
ref	<code>git reflog</code>	Recovery history
conflicts	<code>git diff --name-only --diff-filter=U</code>	Show conflicts
root	<code>git rev-parse --show-toplevel</code>	Repo root

ALIAS CATEGORIES

	STATUS & INSPECTION See what's happening
	HISTORY & LOGS Understand the past
	BRANCHING Create, switch, manage
	STAGING & COMMIT Stage changes, commit cleanly
	REMOTES & SYNC Fetch, pull, push, sync
	SEARCH & REVIEW Find, inspect, compare
	RECOVERY & SAFETY Reflog, stash, restore
	CLEANUP & DIAGNOSTICS Keep repos healthy
	DEVOPS & CI Deterministic & script-friendly

USE WITH CAUTION!



- ⊗ `reset --hard`
- ⊗ `clean -fdx`
- ⊗ `push --force`
- ⊗ `push --mirror`
- ⊗ `branch -D`
- ⊗ `reflog expire`
- ⊗ `gc --prune=now`
- ⊗ `filter-repo`
- ⊗ `rebase -i`
- ⊗ `commit --amend`



Always **inspect**
before you
destroy.
Be explicit.
Be safe.

👥 ALIAS SETS FOR EVERY ROLE



DEVELOPERS

Everyday productivity

- ✓ Status, logs, branches
- ✓ Staging & committing
- ✓ Diff, search, review
- ✓ Recovery & safety
- ✓ Clean, simple, effective



DEVOPS ENGINEERS

Automation & reliability

- ✓ CI/CD friendly aliases
- ✓ Deterministic outputs
- ✓ Repo diagnostics
- ✓ Health & integrity
- ✓ Script-ready commands



SAFETY FIRST

Protect your work

- ✓ Preview before delete
- ✓ Prefer reversible actions
- ✓ Use `--force-with-lease`
- ✓ Backup before rewrite
- ✓ Recovery is possible



MAKE GIT YOURS.

Aliases turn
complex commands
into muscle memory.

**Work smarter.
Ship faster.**



COMPLETE 35-PART GIT COMMAND REFERENCE

- | | | |
|-------------------------|---------------------------|-----------------------------|
| 1 Configuration | 13 Searching History | 25 Git Objects / Internals |
| 2 Creating Repositories | 14 Comparing Branches | 26 Ignoring Files |
| 3 Status & Information | 15 Conflict Resolution | 27 File Tracking |
| 4 Staging & Committing | 16 Cherry-Pick | 28 Sparse Checkout |
| 5 Diff & Code Review | 17 Reflog & Recovery | 29 Git Archives |
| 6 Branching | 18 Git Bisect | 30 Environment & Automation |
| 7 Merging | 19 Submodules | 31 DevOps / CI Commands |
| 8 Rebasing | 20 Worktrees | 32 Common Workflows |
| 9 Remote Repositories | 21 Git LFS | 33 High-Value Commands |
| 10 Undoing Changes | 22 Git Hooks | 34 Dangerous Commands |
| 11 Stash | 23 Repository Maintenance | 35 Practical Git Aliases |
| 12 Tag & Release | 24 Repository Diagnostics | |



★ ★ ★ ★ ★
YOU MADE IT!
35 CHAPTERS.
MASTER GIT.
BUILD ANYTHING.

KEEP LEARNING. KEEP BUILDING.

